KAZAKH BRITISH TECHNICAL UNIVERSITY

School of Information Technology and Engineering

Automated Control Systems Based on Simatic S7-1200

Checked by: Svambayeva A.S

Done by: Yespembetov Mukhammed Ali

Almaty 2025

PRACTICAL WORK NO. 8. DESIGNING LOGIC AND CALCULATIONS USING THE FBD(FUNCTIONAL BLOCK DIAGRAM) LANGUAGE

Purpose of the practical work:
1. Familiarization with principles construction logics in functional block language (FBD).
2. Studying the structure of an FBD program and the logic of networks.
3. Mastering work with basic types FBD-blocks: logical, arithmetic, control.
4. Gain skills in building and testing programs in TIA Portal for Siemens S7- 1200 controller.

8.1 Basic theoretical information:

FBD (Function Block Diagram) is a graphical programming language used to develop custom algorithms for PLCs (Programmable Logic Controllers). It is one of the standard languages defined in the international standard IEC 61131-3. Its main advantage is the visual representation of logic in the form of interconnected functional blocks, which makes it easier to understand and debug programmed, especially when working with complex control systems.
A program in FBD consists of blocks (Blocks), each of which performs a certain function - logical, arithmetic, control or specialized. Blocks are combined into networks. Each network contains one or more computational expressions and ends with an assignment block, which defines the output result of calculations.

8.1 Table of element type

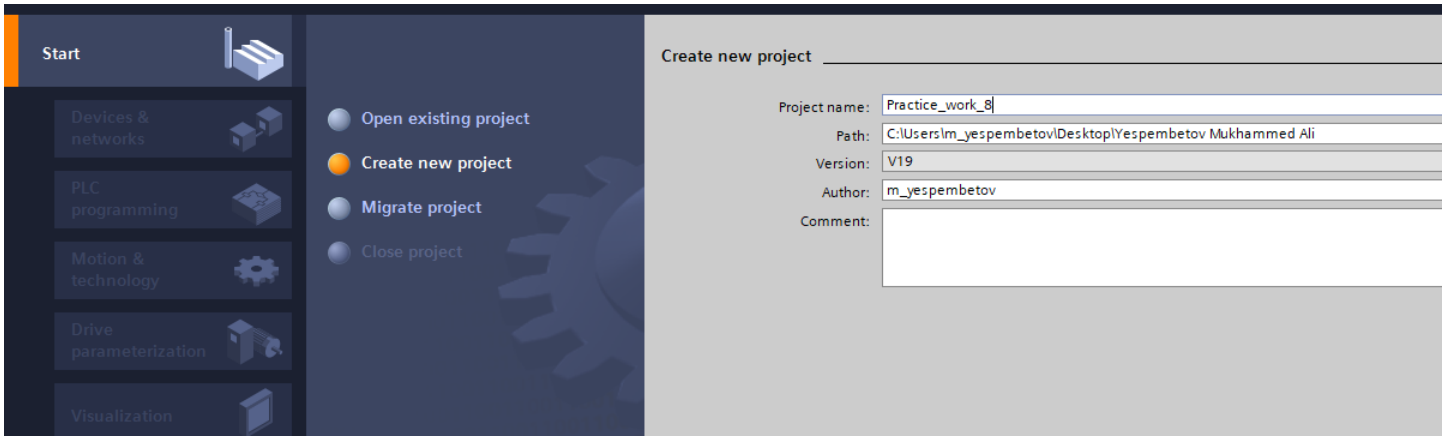
Element type	Description and examples
Binary functions	logical operations AND, OR, XOR. Input signals are scanned to logic level (1 or 0). Applies to boolean variables and pins.
Standard boxes	Used to store the result of logic and pass it to the output. Often terminate the network via assignment to a tag
Boxes with Q output	Memory blocks, timers, counters. They have Q output, which determines the result of operation. Example: SR (Set-Reset trigger).
Boxes with EN/ENO input/output	Entry EN activates operation unit, output ENO reportssuccessful processing. They are used in arithmetic, type conversions (REAL⇄ INT).
Block calls (FC/FB)	Calls of program blocks with parameters. Used for structuring and reusing logic.

Processing of normally open (NO) and normally closed (NC) contacts. The FBD language provides the possibility of equal processing of signals coming from both normally open (NO) and normally closed (NC) contacts. This is achieved by selecting the appropriate type of input element in the block.

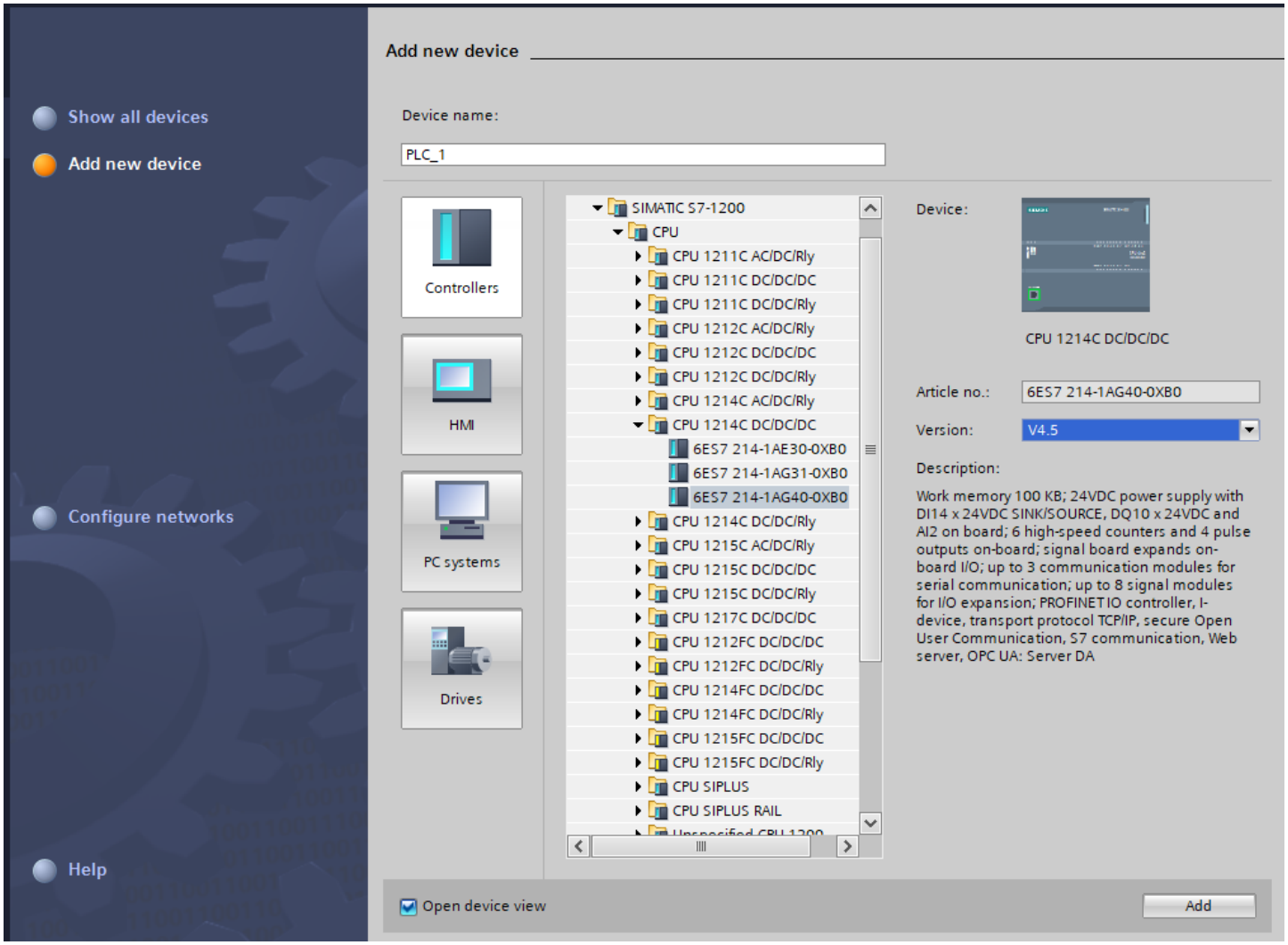
Task 1: In this task you need to create a program to control traffic lights with three light signals: red, yellow and green. The traffic light operation is implemented using three timers that sequentially activate the corresponding signals. It is also necessary to visualize the traffic light operation on the HMI panel.

Step-by-step implementation

- **(Step 1)** Open TIA Portal --> Click “Create new project.” and write «Project name». Then click «Create»



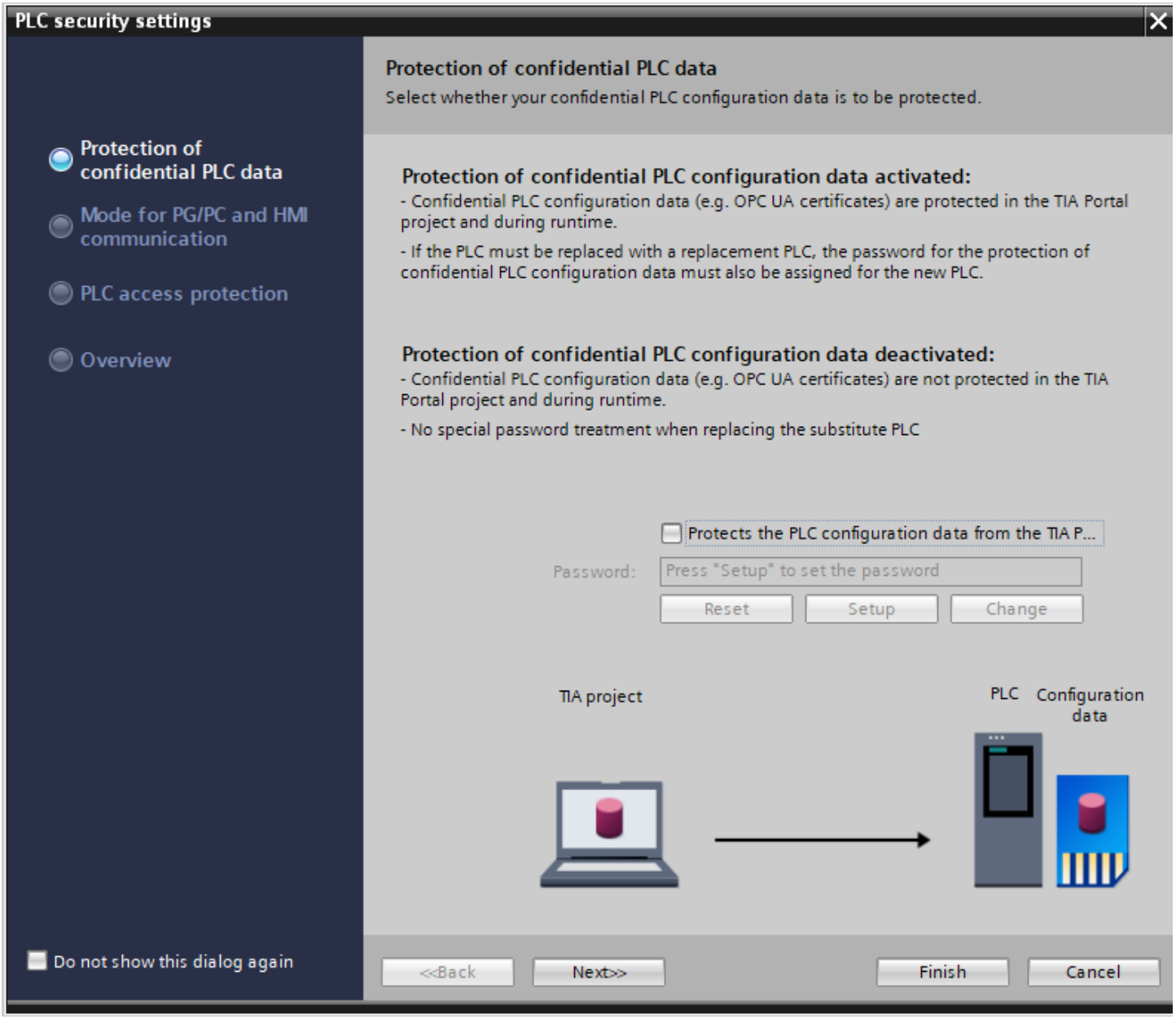
- **(Step 2)** Then, from the left panel, select 'Device & Networks' and choose 'Add new device'. Then, open the «SIMATIC S7-1200» → «CPU» → «CPU1214C DC/DC/DC» and select «6ES7 214-1AG40-0XB0», so click the «Add» button.

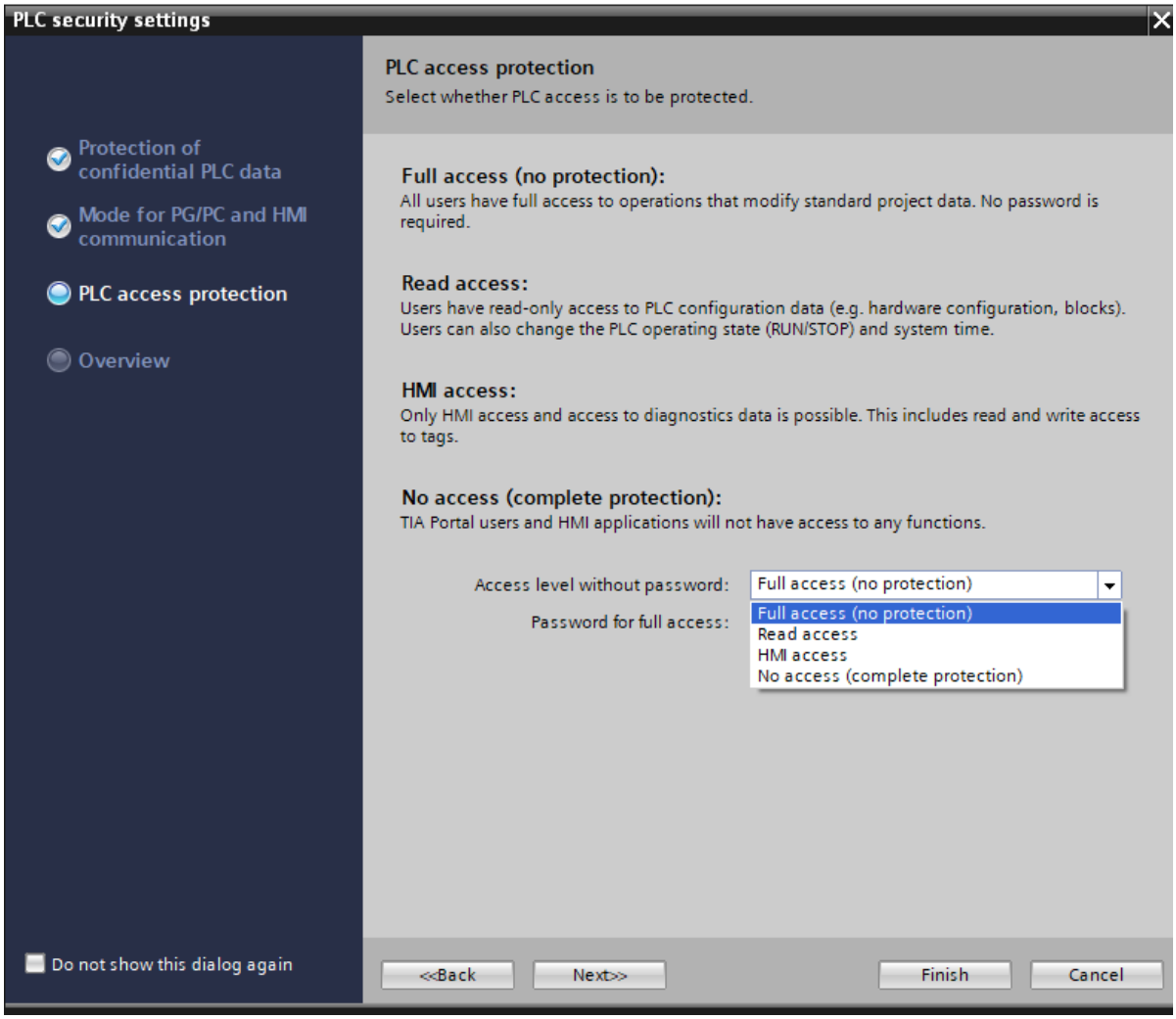


Next you will be presented with the controller security settings

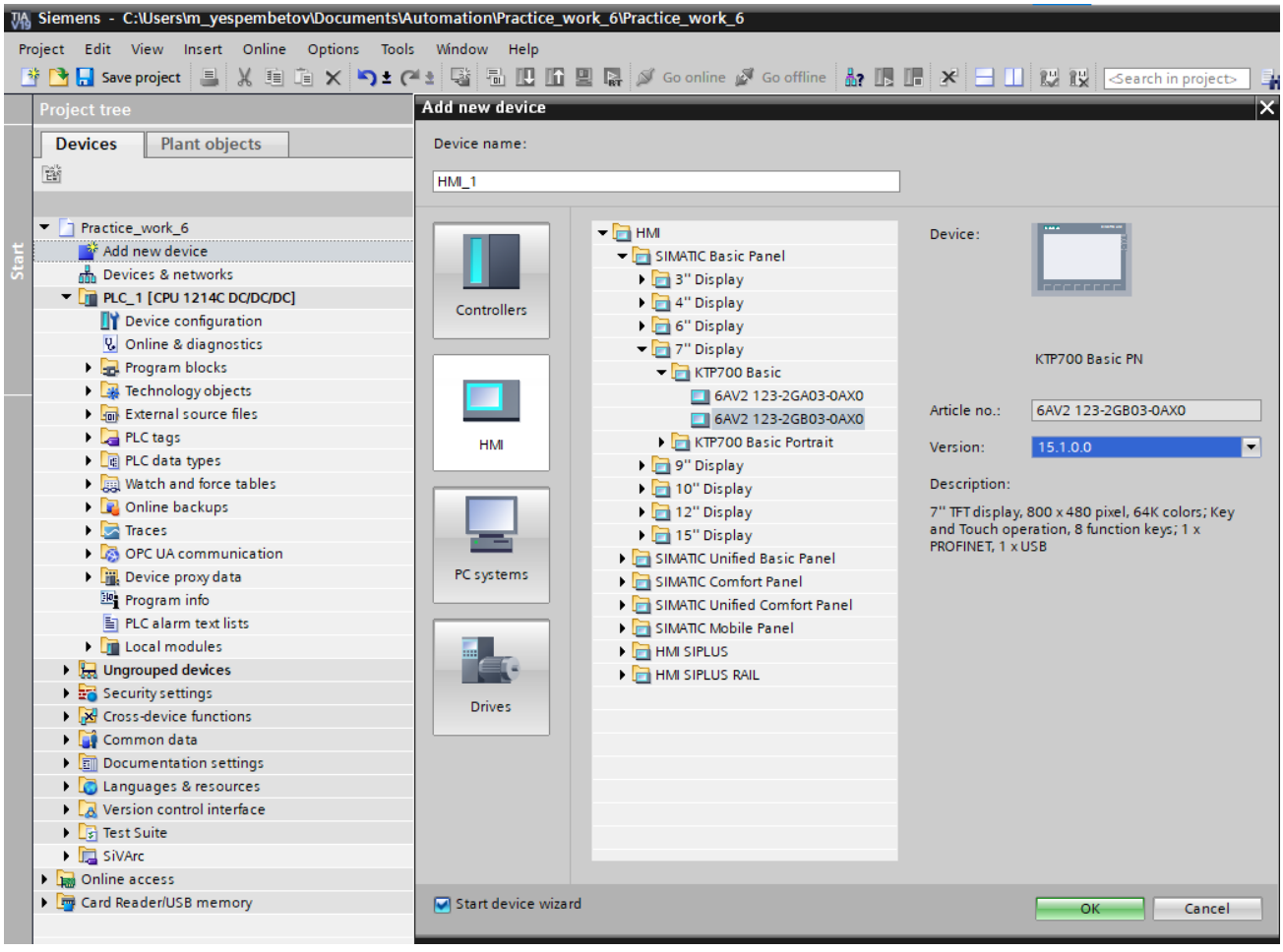
In the opened window you will see the following permissions to control and re-flash the controller. Uncheck **"Protects the PLC configuration data from the TIA Portal project and the PLC"**, **"Only allow secure PG/PC and HMI communication"**. Each will be on a separate window.

On step 3, when selecting the access level without password, set **"Full access (no protection)"** and then click on **"Finish"**.

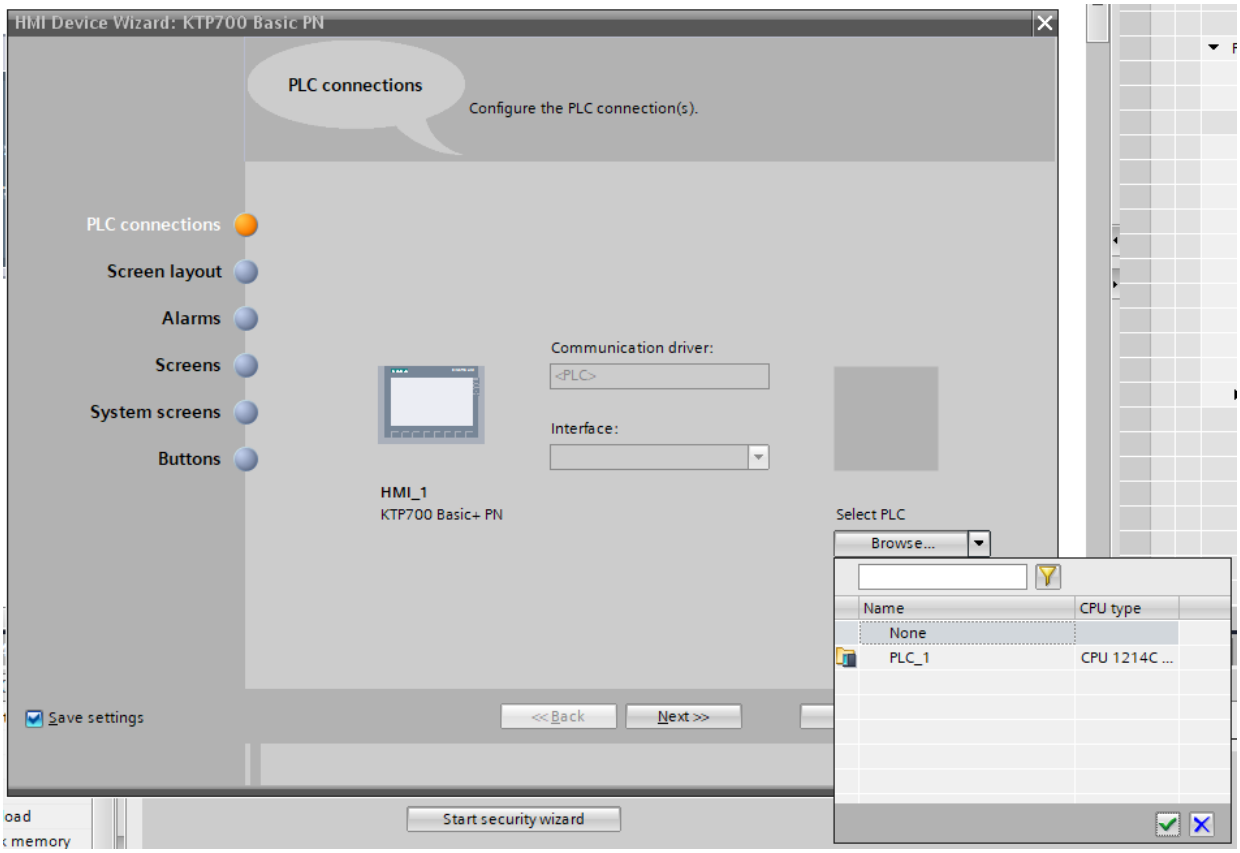


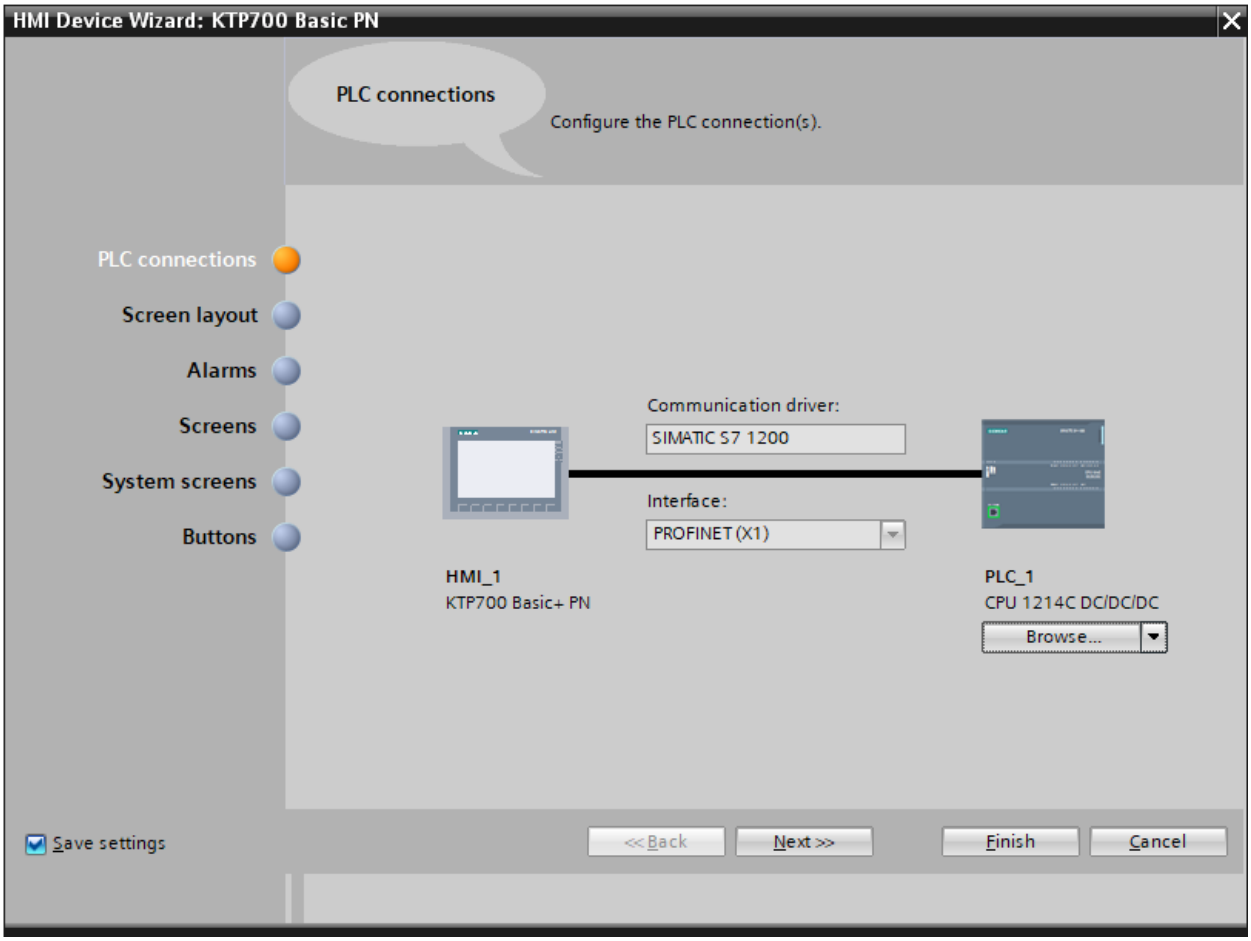


Step 4 Connecting HMI to TIA Portal

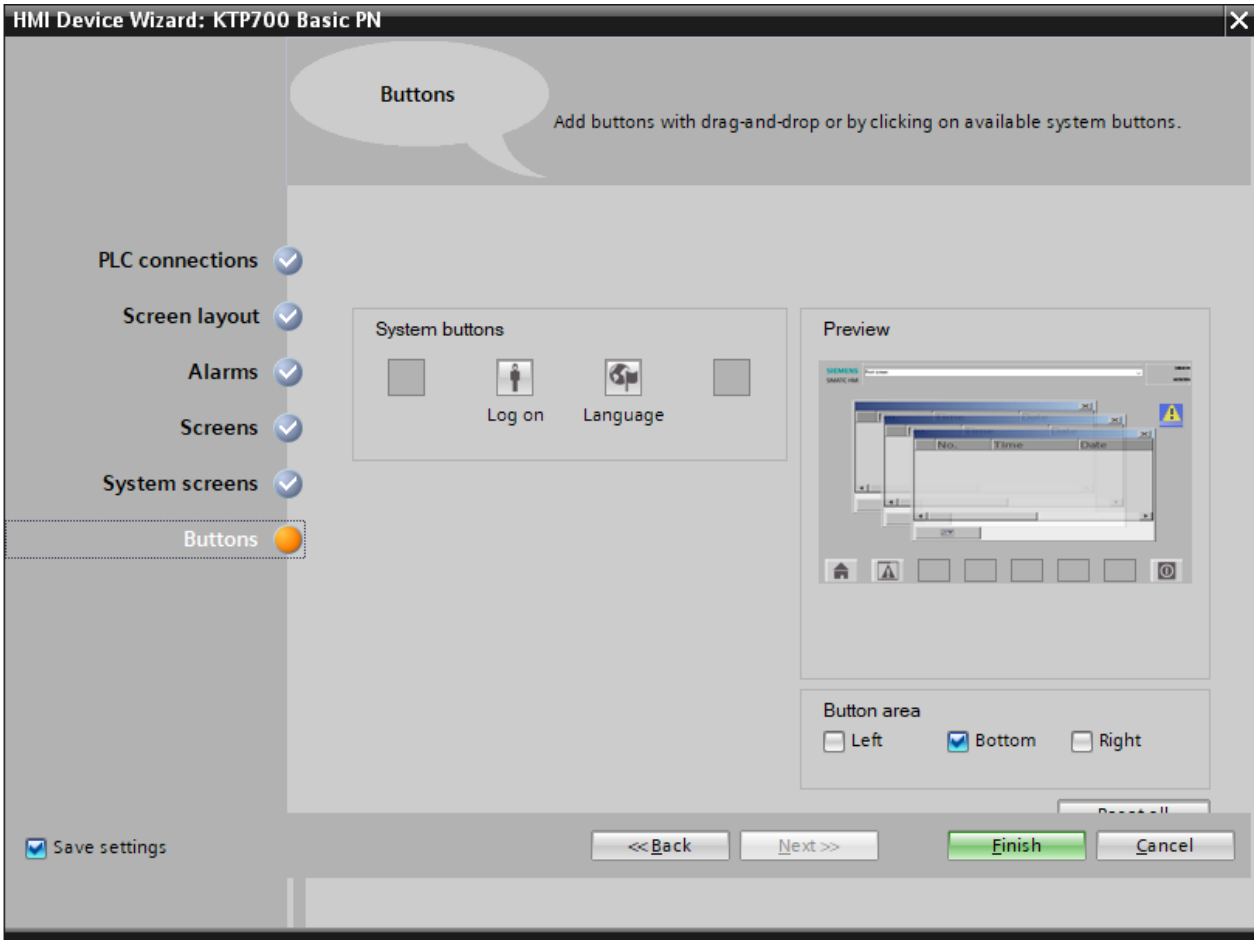


Step 5 After selecting the appropriate device, you need to connect it to your controller. Both devices must be on the same subnet. In this case, the IP address indicated on the controller's nameplate is used.When selecting the HMI firmware version, it is necessary to set 15.1.0.0. If the correct HMI model is selected, but the firmware is different, an error about firmware mismatch will appear when loading the project. If an error occurs, it is better to ask the engineer to check the correctness of the actions performed.On the window that will appear after adding HMI. It is necessary to connect to the controller. Click on the "Browse" button and a list of available devices in the TIA Portal will appear at the bottom. Choose your controller among them.



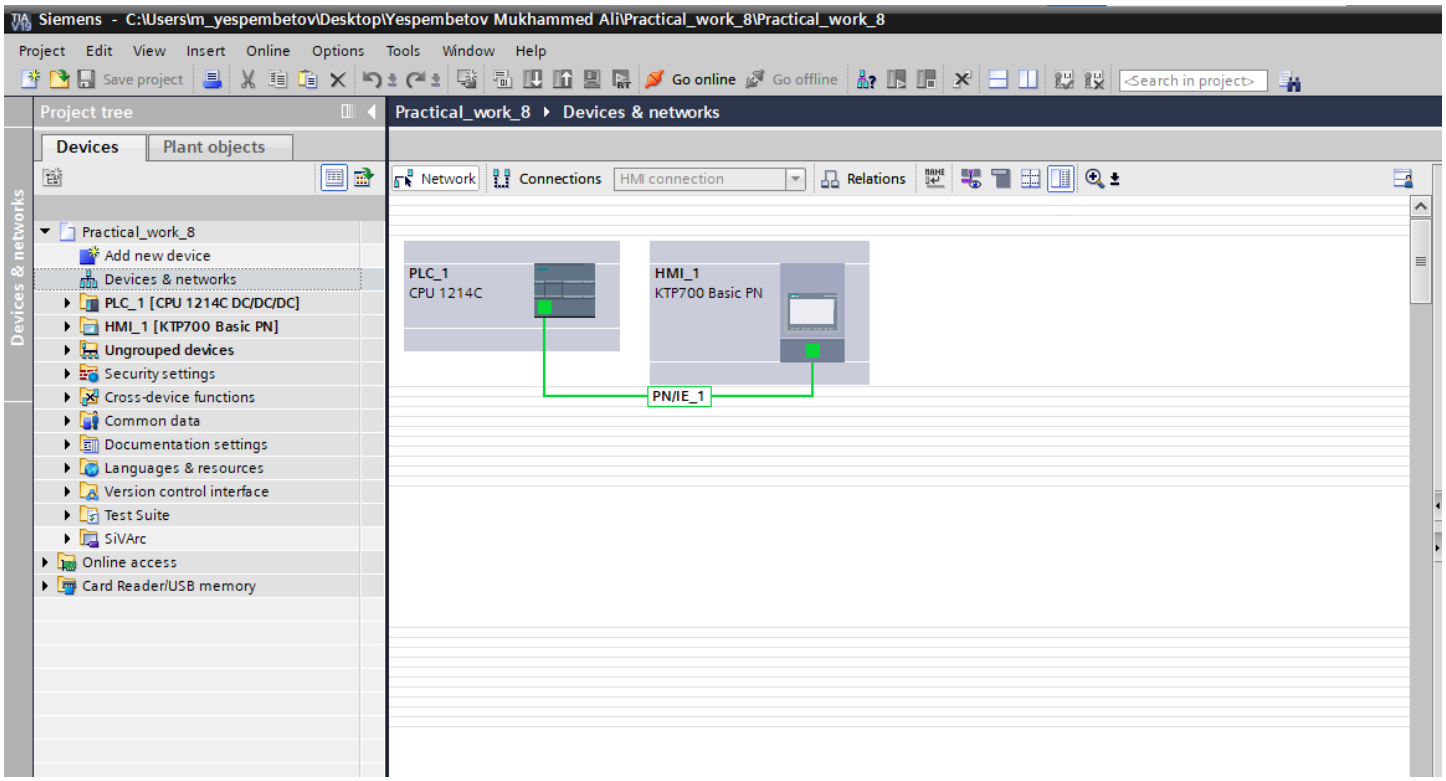


After connection a line between the controller and HMI should appear

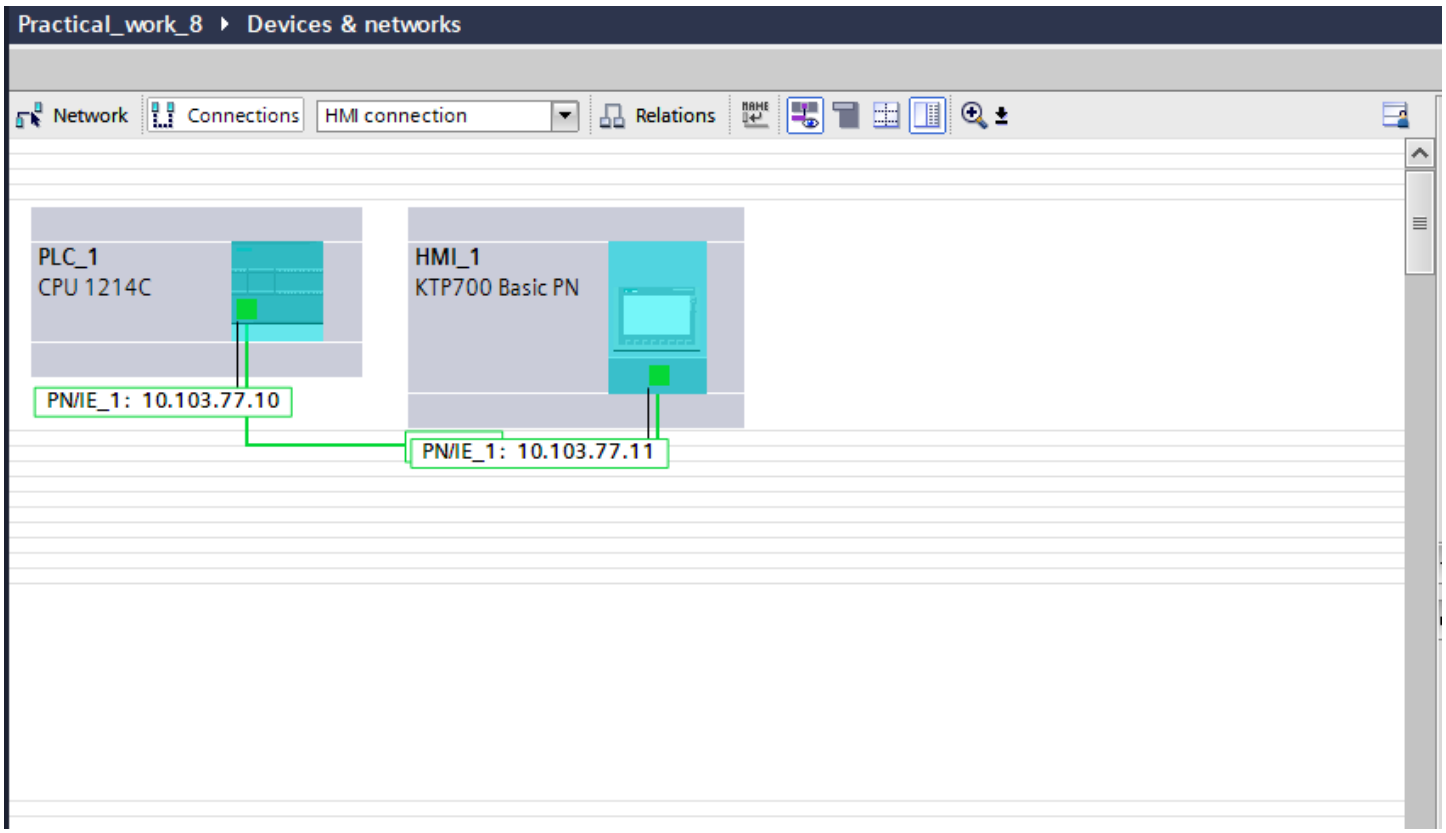


Once the connection is made, click on the "Finish" button. You can also check the connection via the "device and networks" button

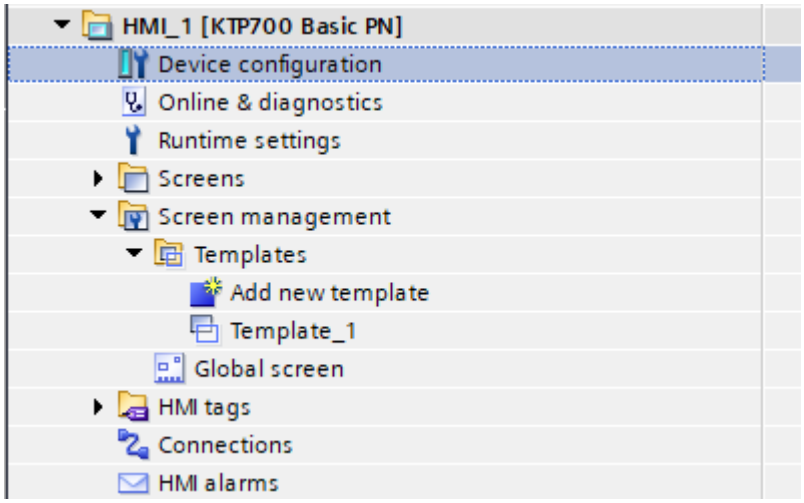
After pressing the button, select show address labels or the icon with an eye in the corner . If the connection failed, create it manually. Select the Ethernet port in this window and connect to the Ethernet port of the HMI.



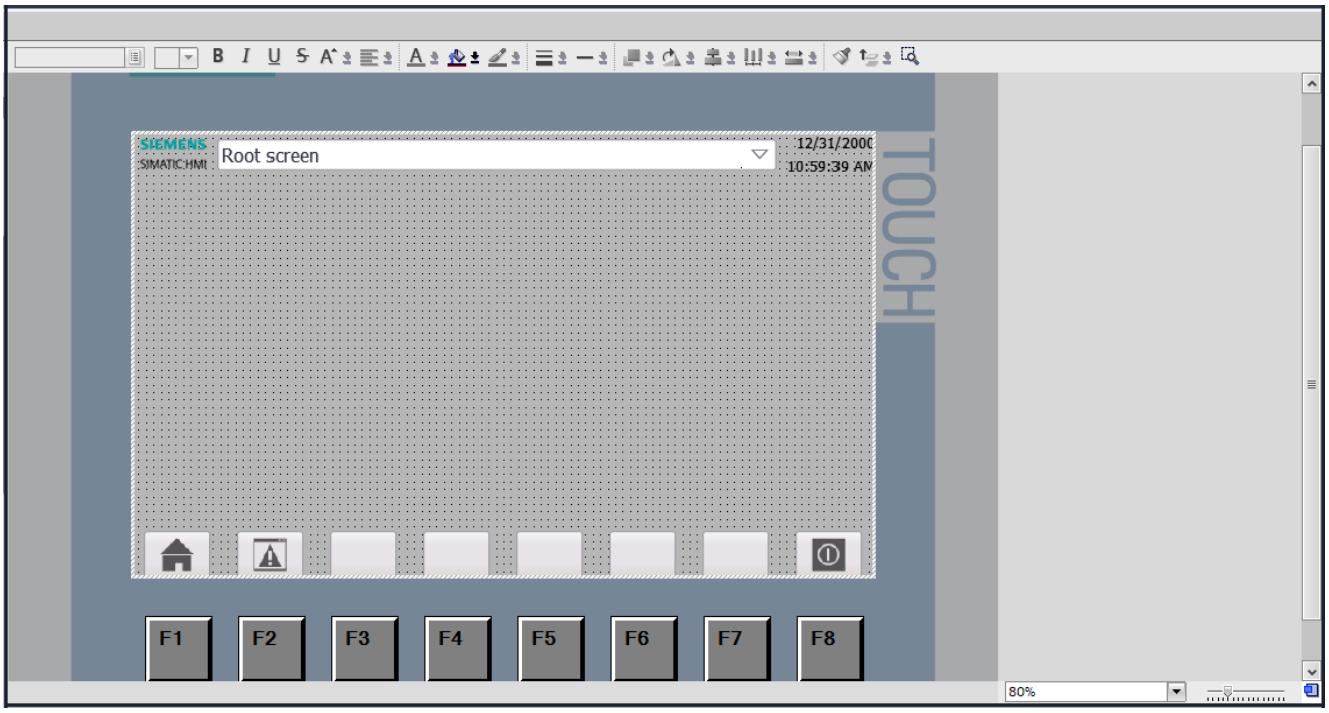
After pressing the button, select show address labels or the icon with an eye in the corner . If the connection failed, create it manually. Select the Ethernet port in this window and connect to the Ethernet port of the HMI.



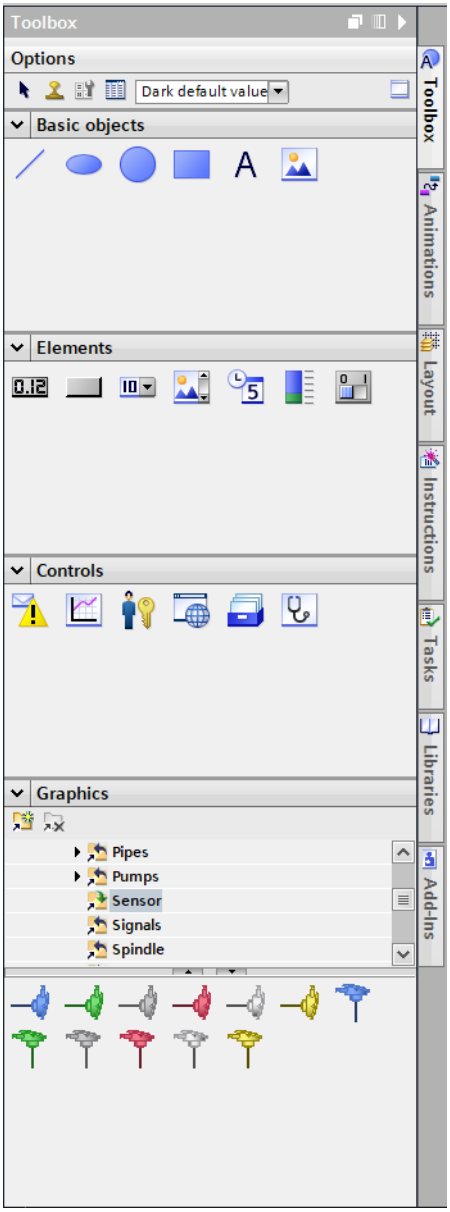
To work with the HMI we go to the template.



After this step in TIA Portal a window will open where you can change different variables or create new ones.



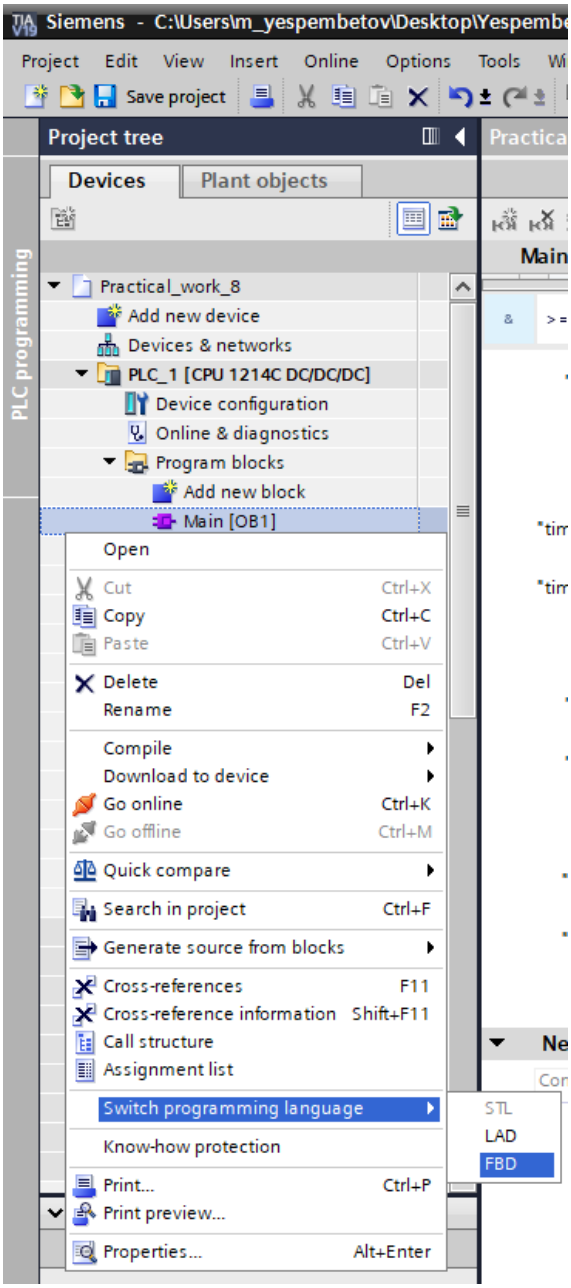
Using the Toolbox window, we display the task model in the "Template_1" section.



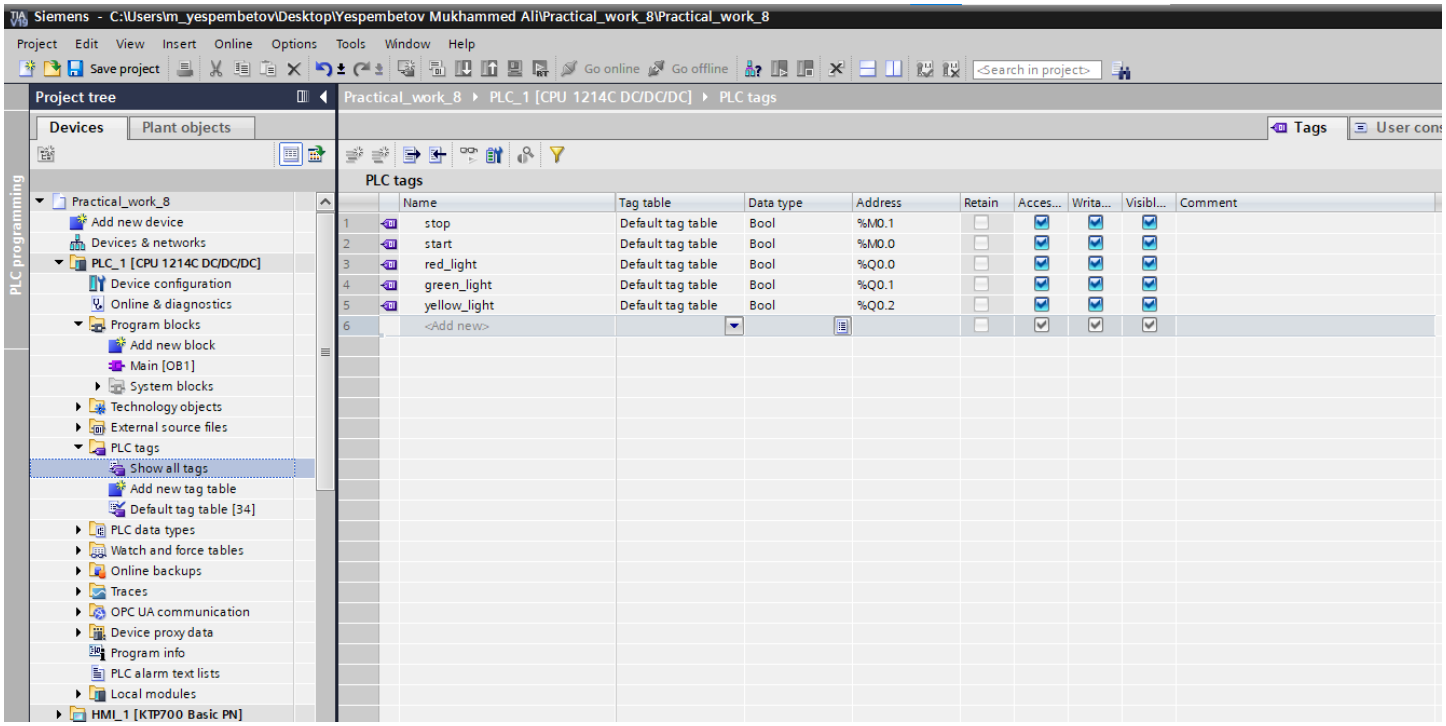
We visualized the conveyor scheme on the HMI panel and arranged the actual control buttons.

The main block in the project.

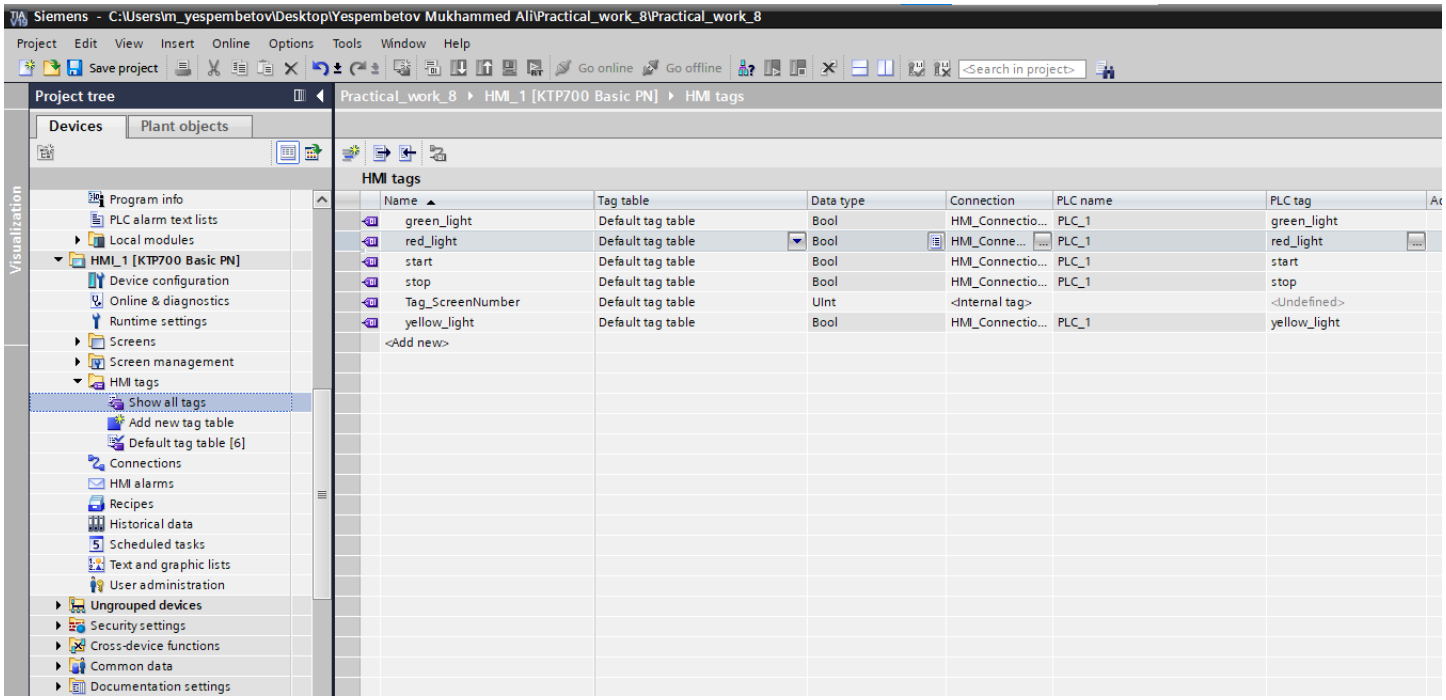
Before writing in the Main block, we need to change it from LAD to FBD language. To do this, right-click and select **“Switch programming language”**, then choose the **FBD** language.



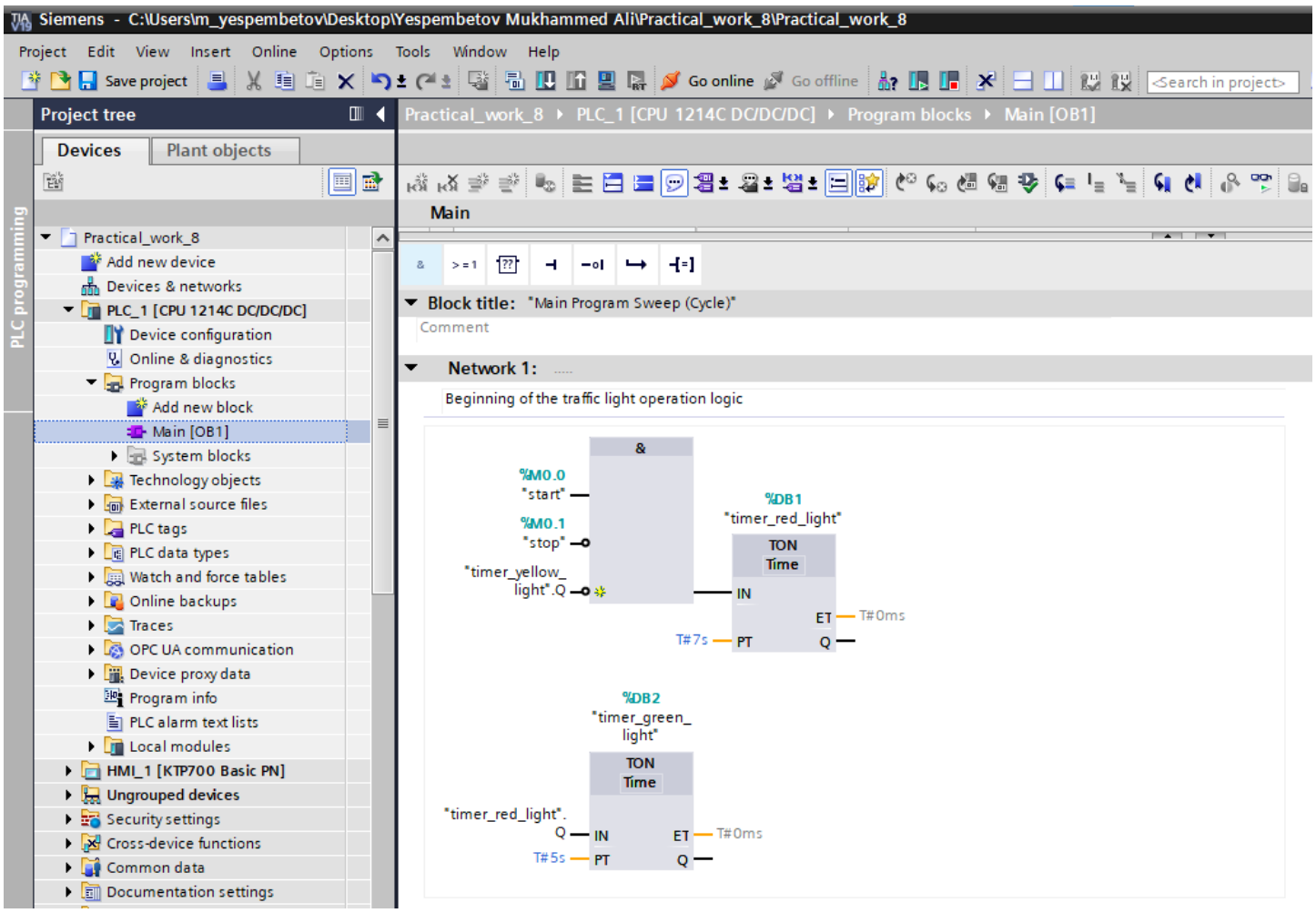
This is our PLC tags, that we will need



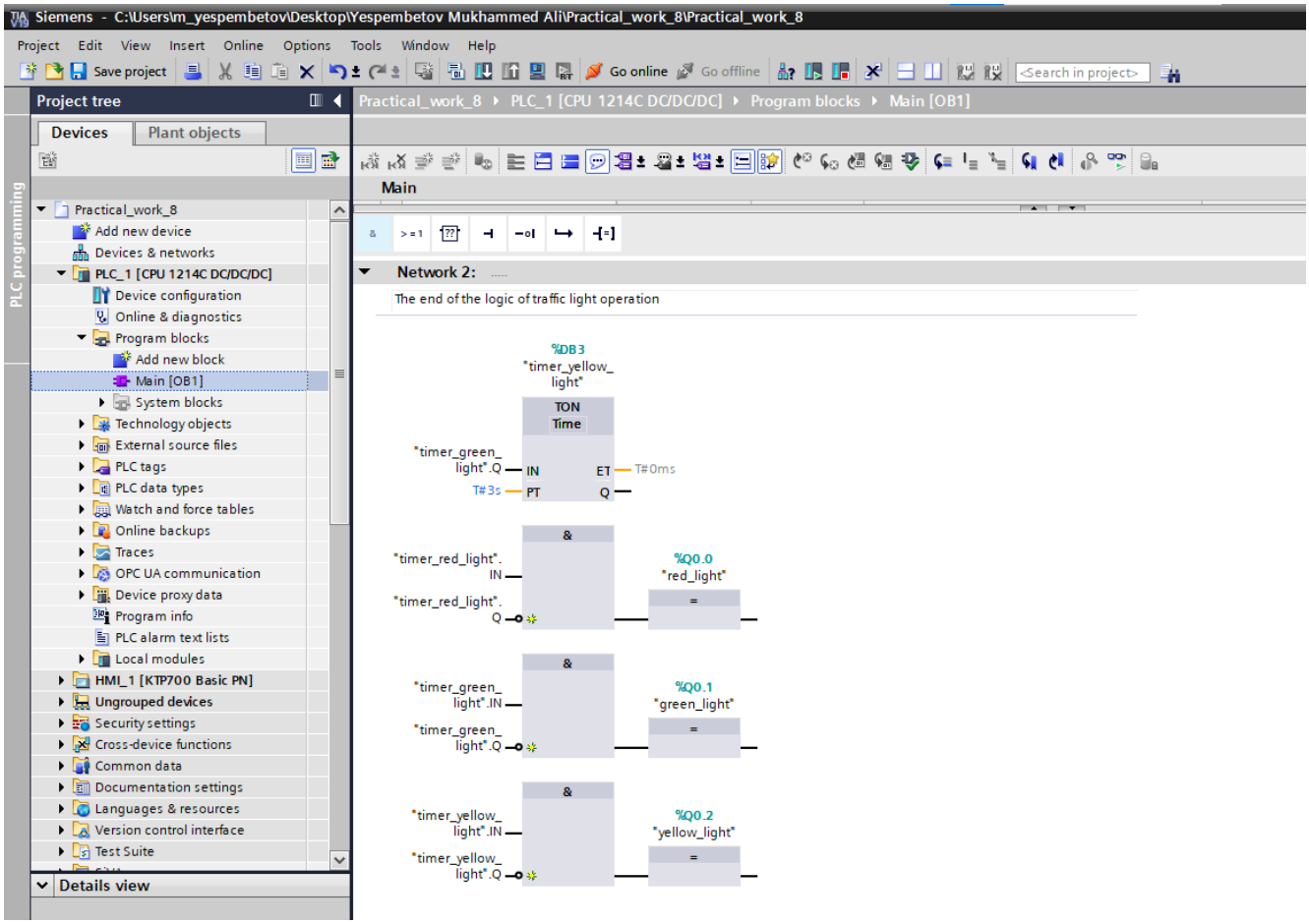
And we can connect HMI tags with PLC tags.



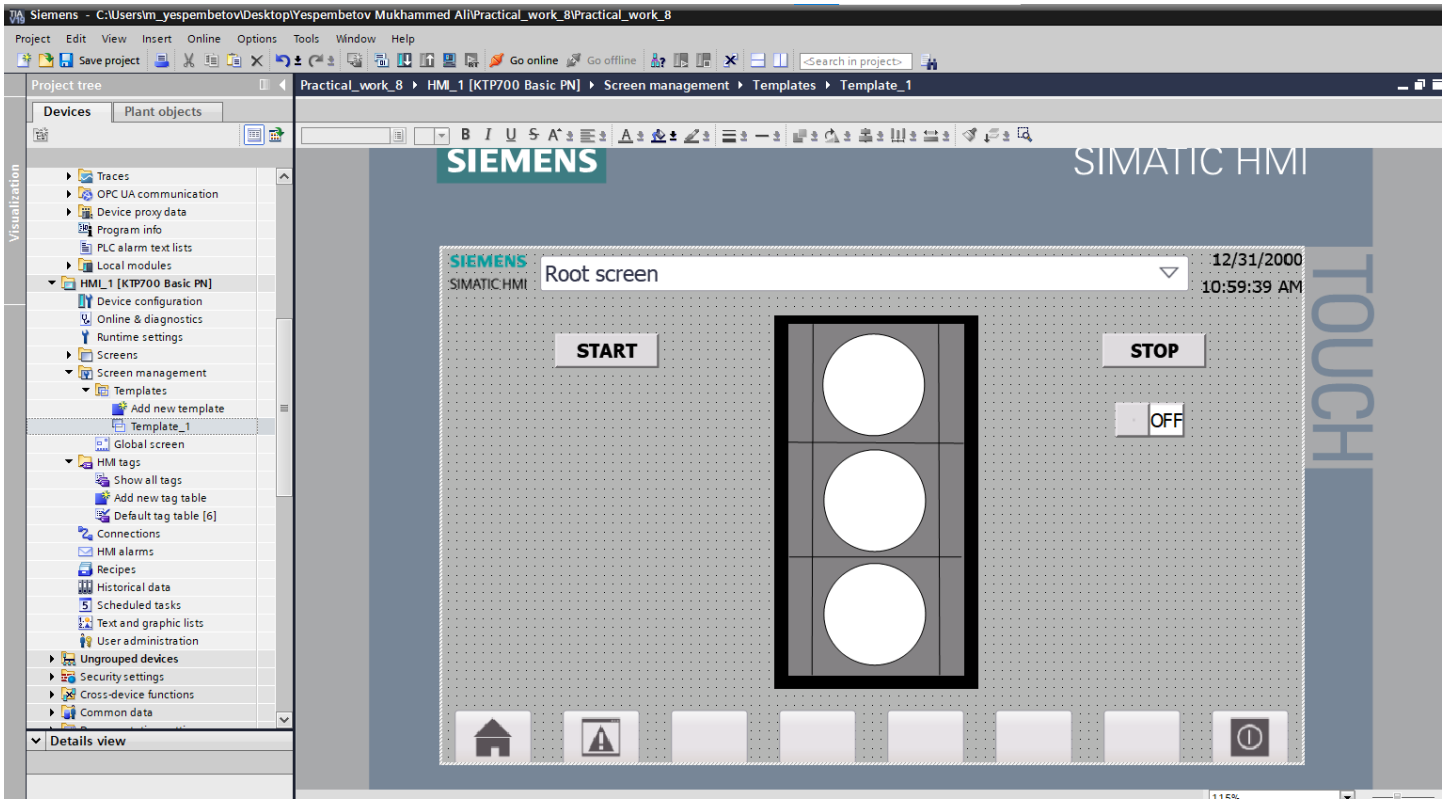
Now we can complete the task in FBD language. Network 1 is beginning of the traffic light operation logic. We use TON timer with 7 s.



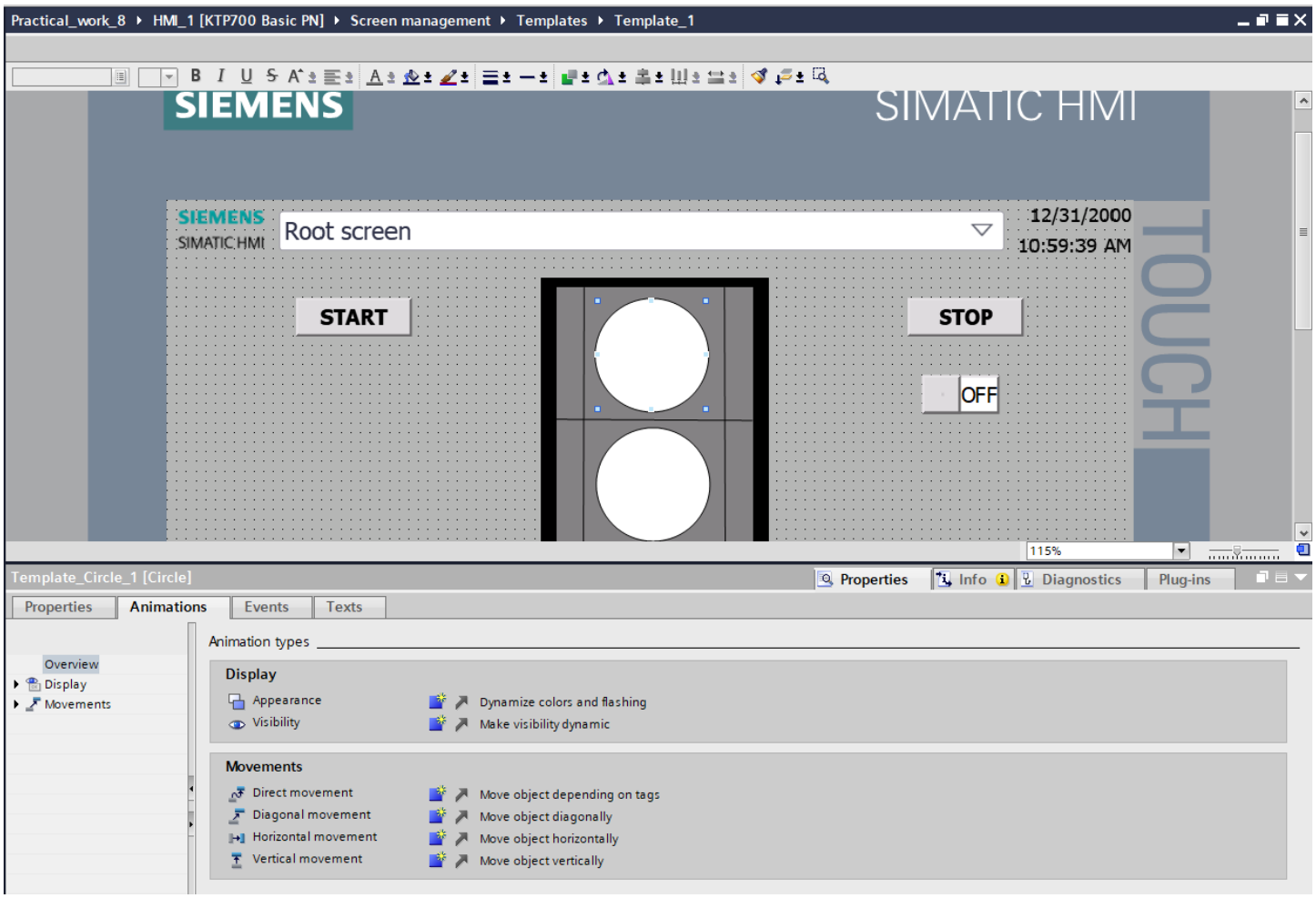
Network 2 it is possible to control the traffic lights



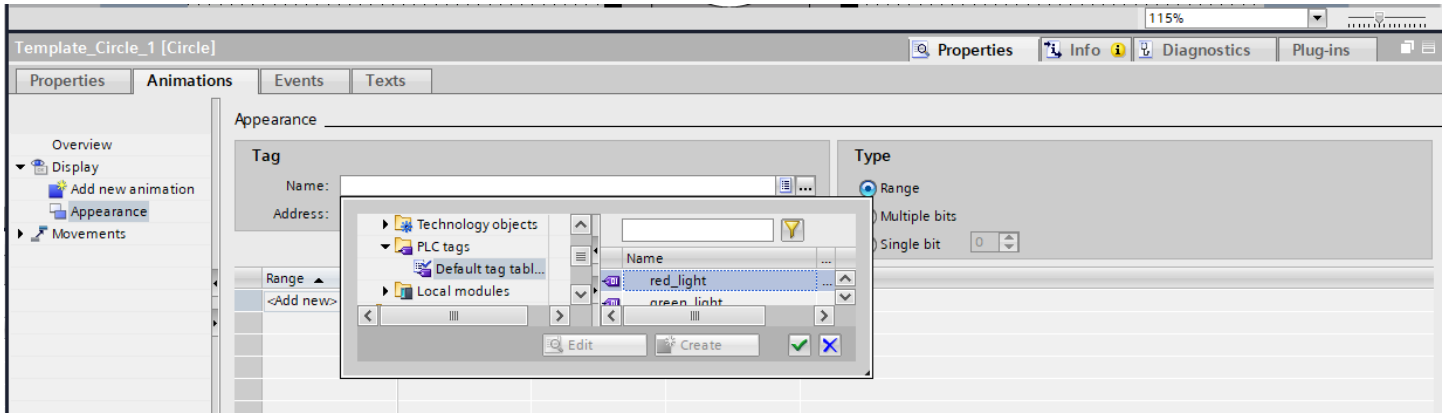
Here we have displayed the traffic light on the HMI, now we need to link it with the tags.



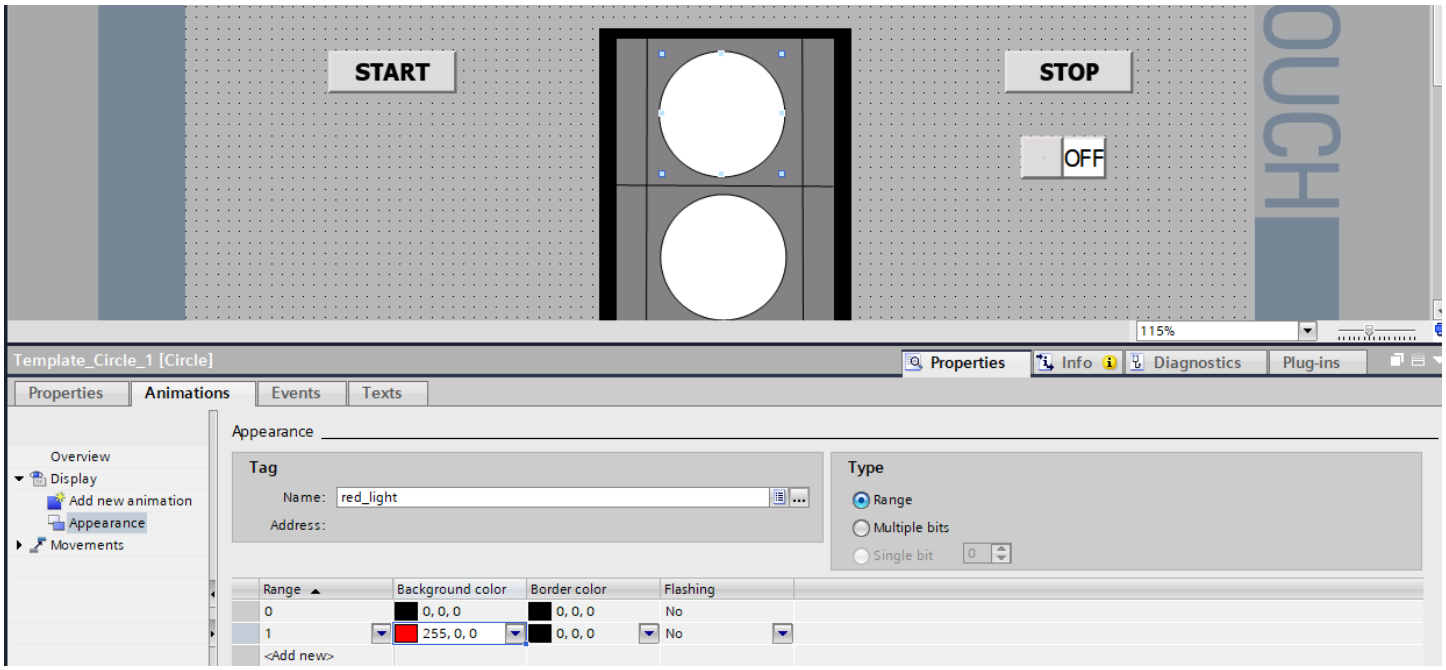
Properties -> Animation -> Choose “Dynamize colors and flashing”



We need to connect with PLC tags

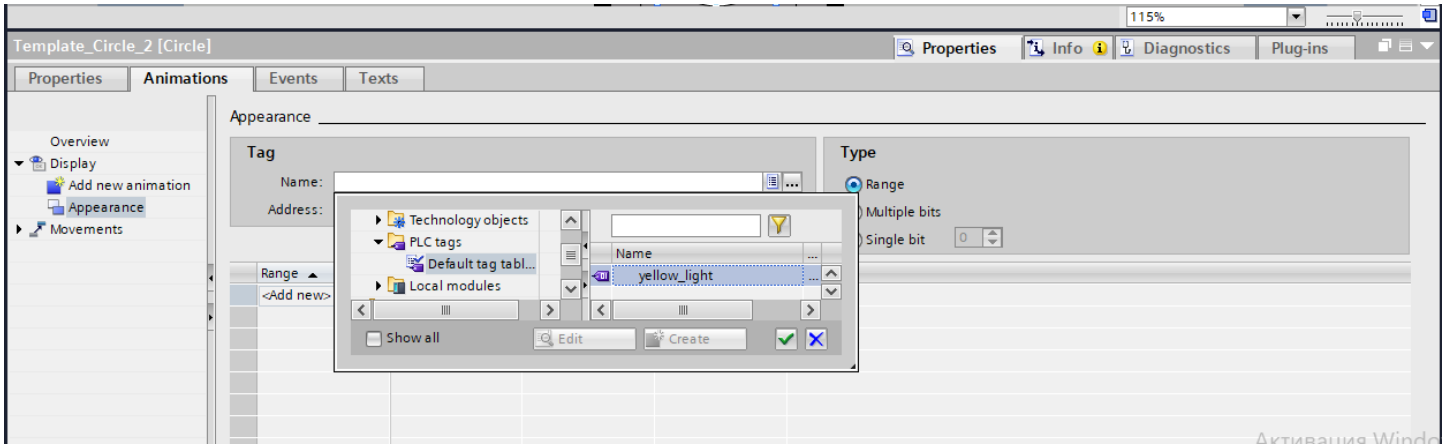


Therefore, we can change colors: 0 is at the beginning, 1 it is our light (if will be ON)

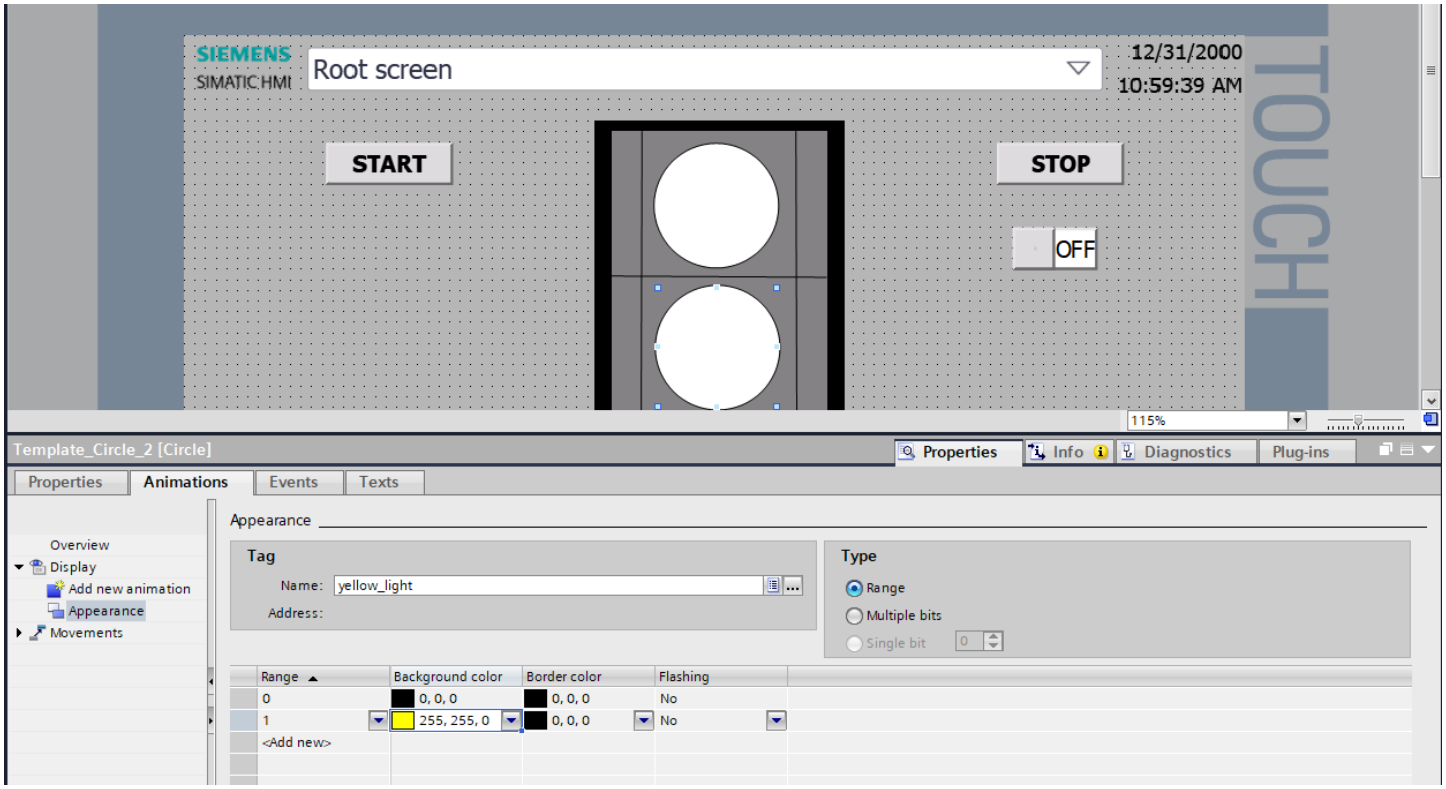


Olso repeat for the yellow lights

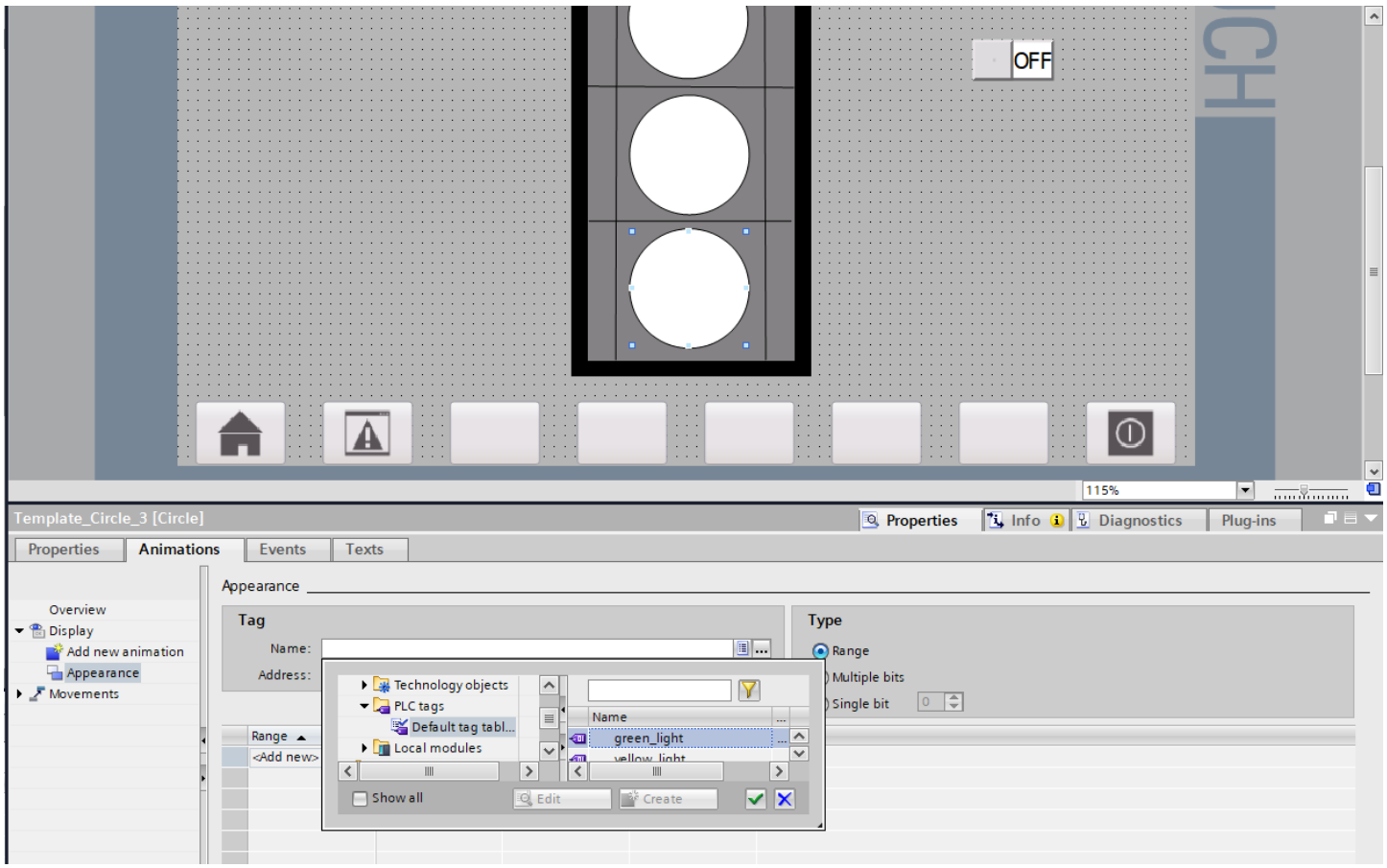
\

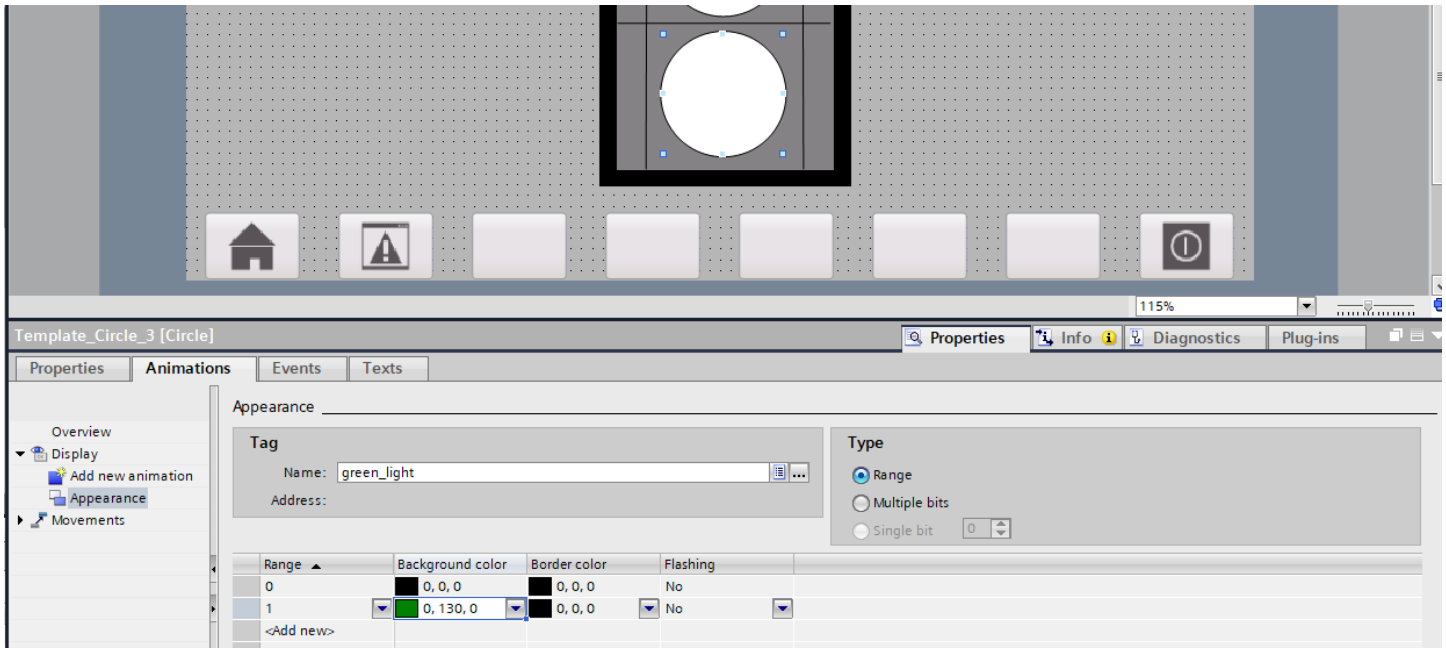


Change colors for each mode

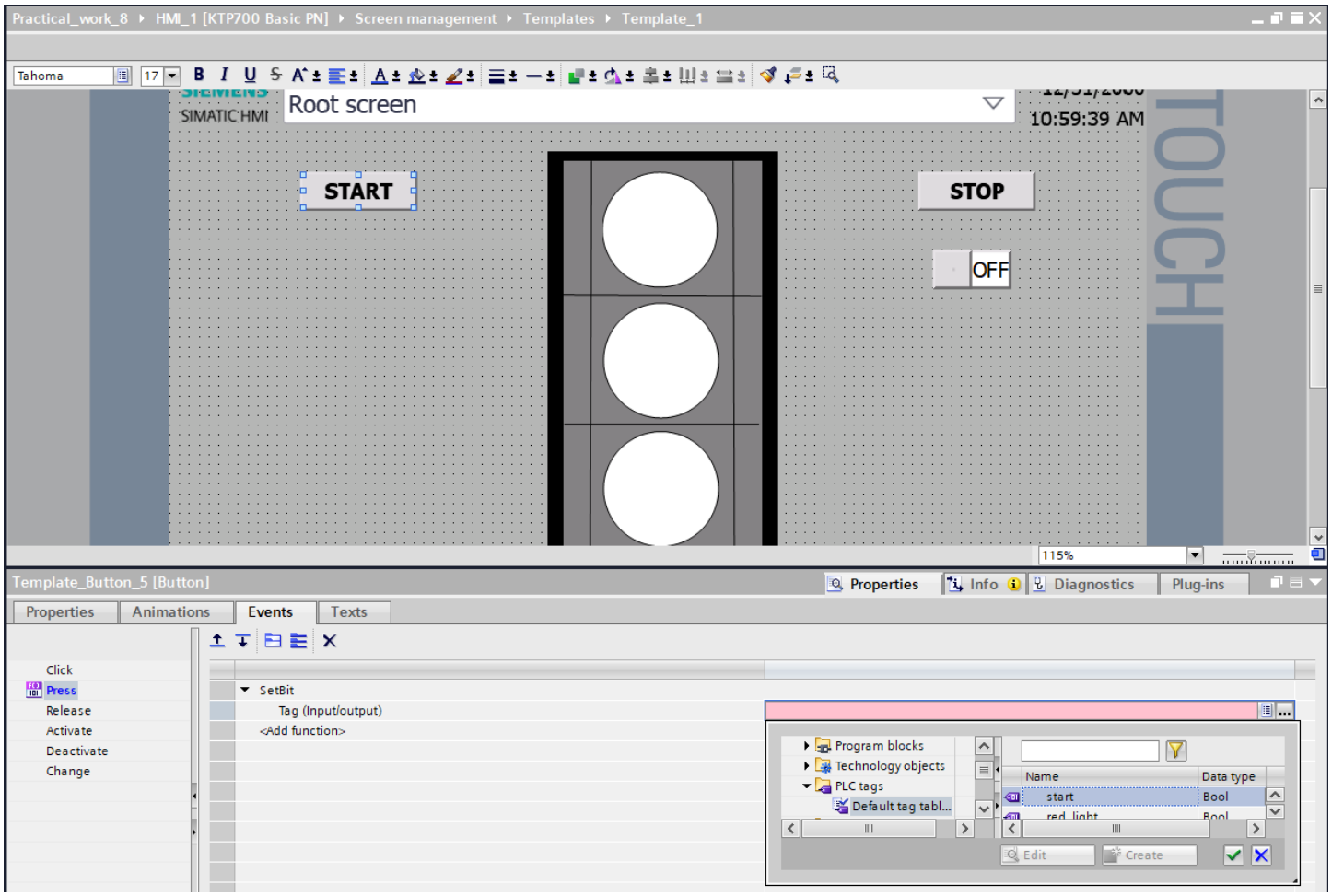


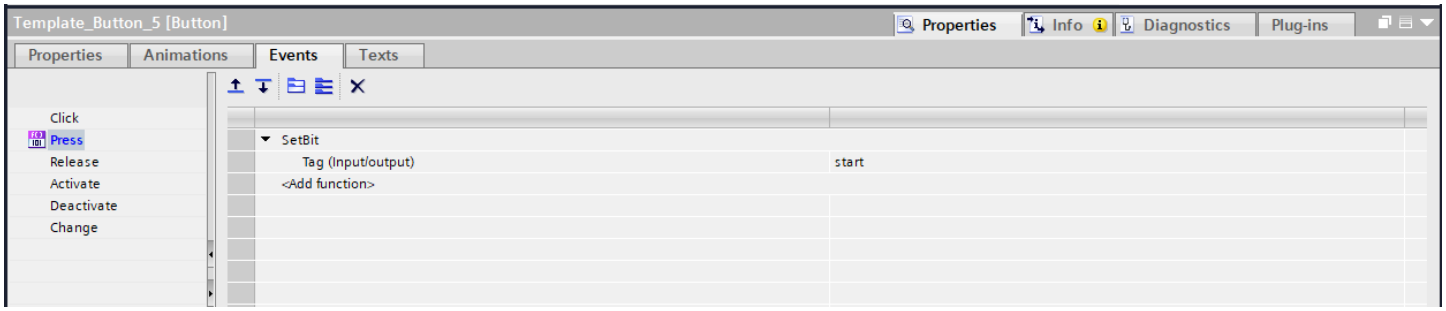
For the green light look at the



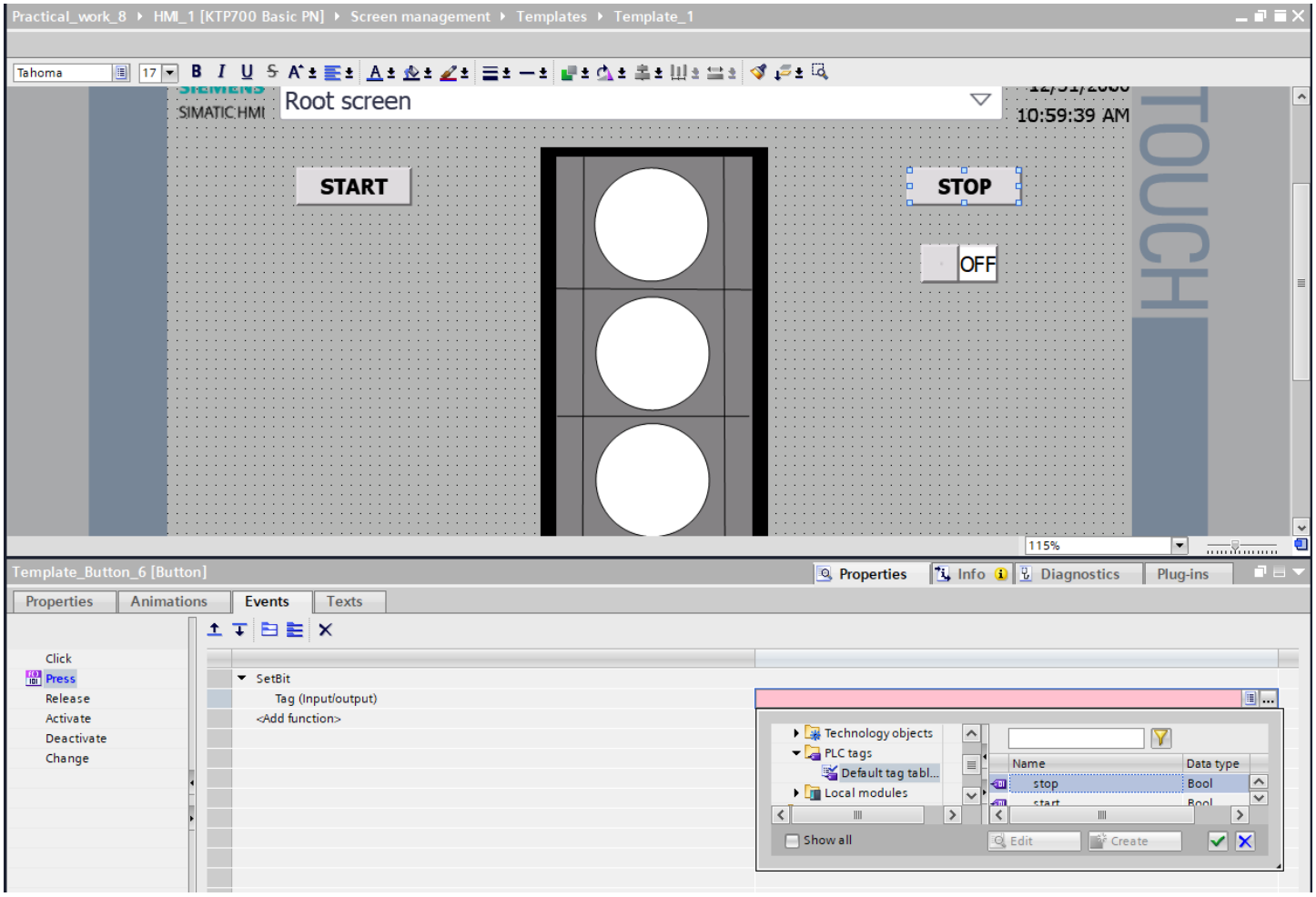


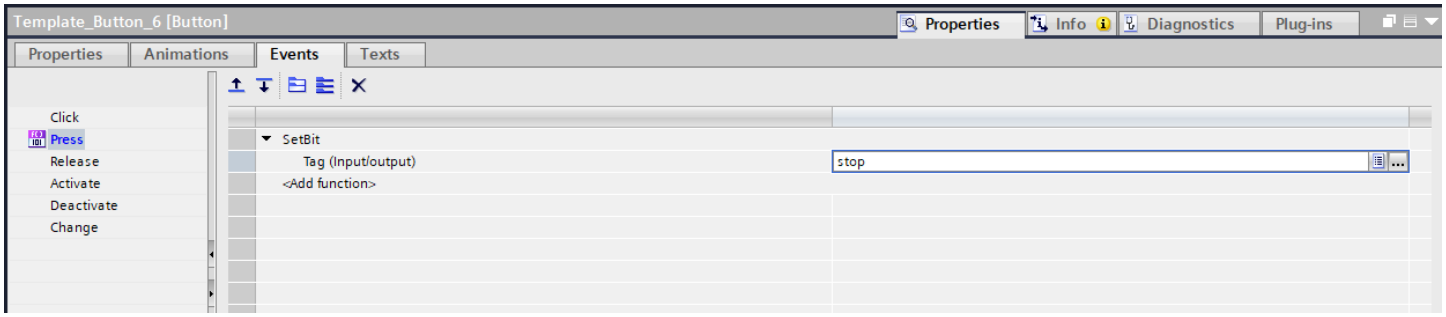
We need to link our important “Start” button with PLC tags



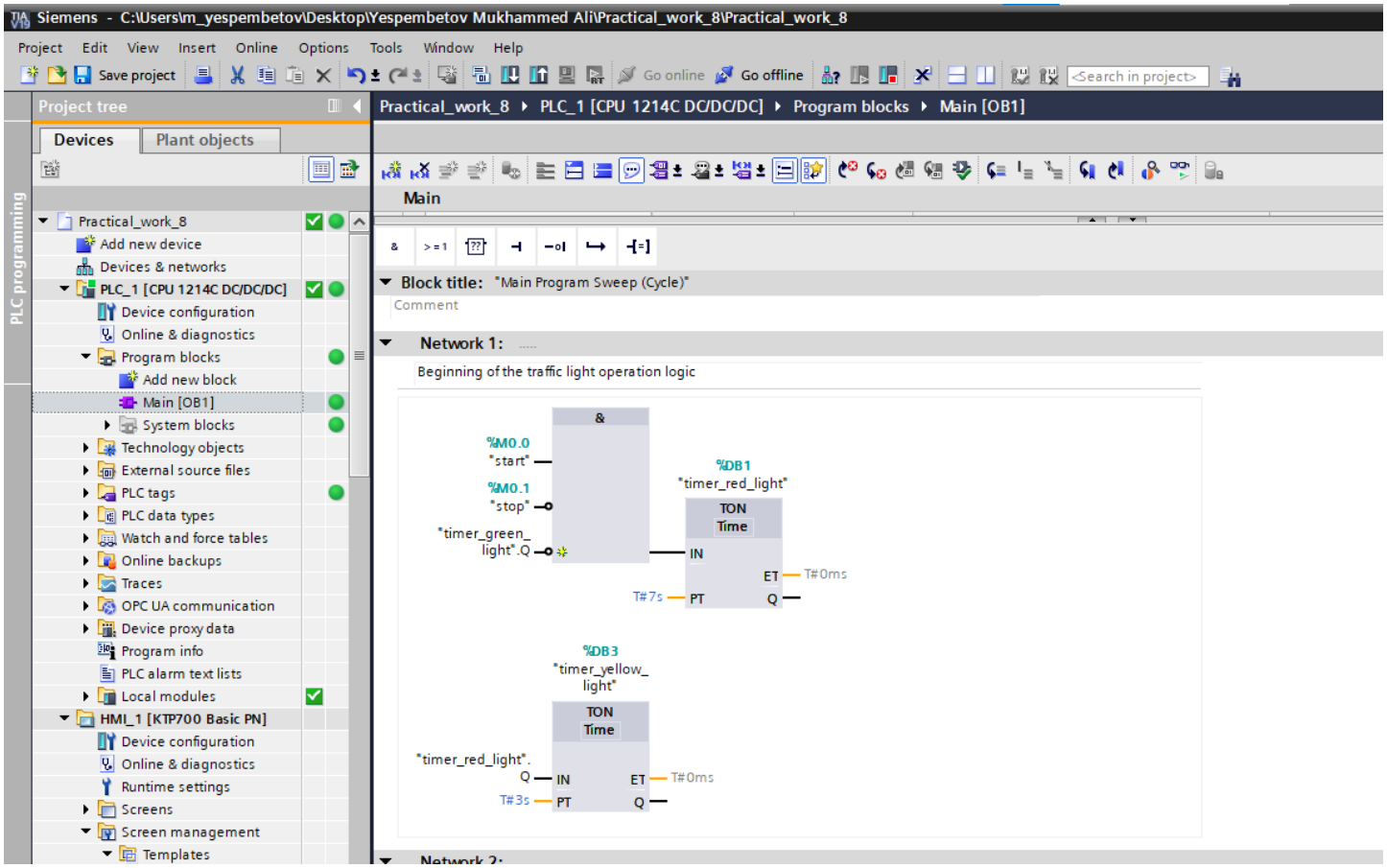


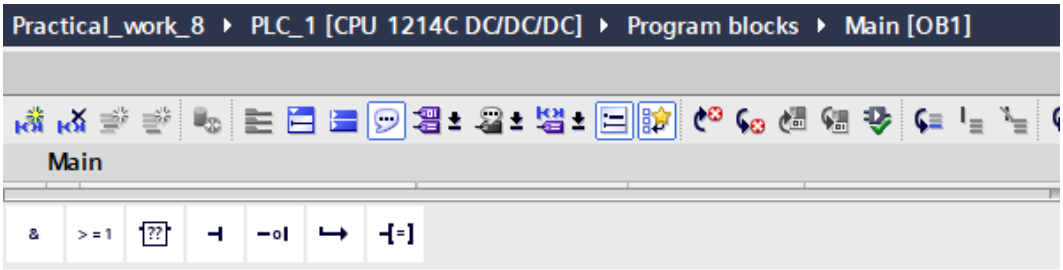
Olso “STOP” button

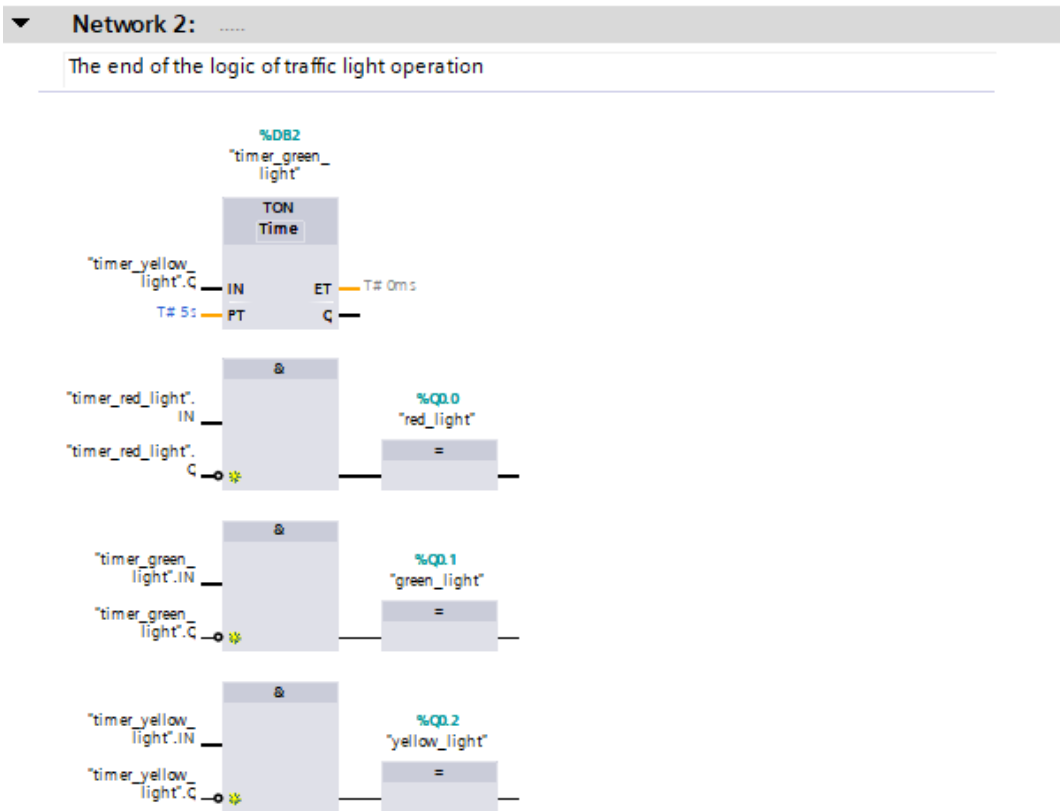




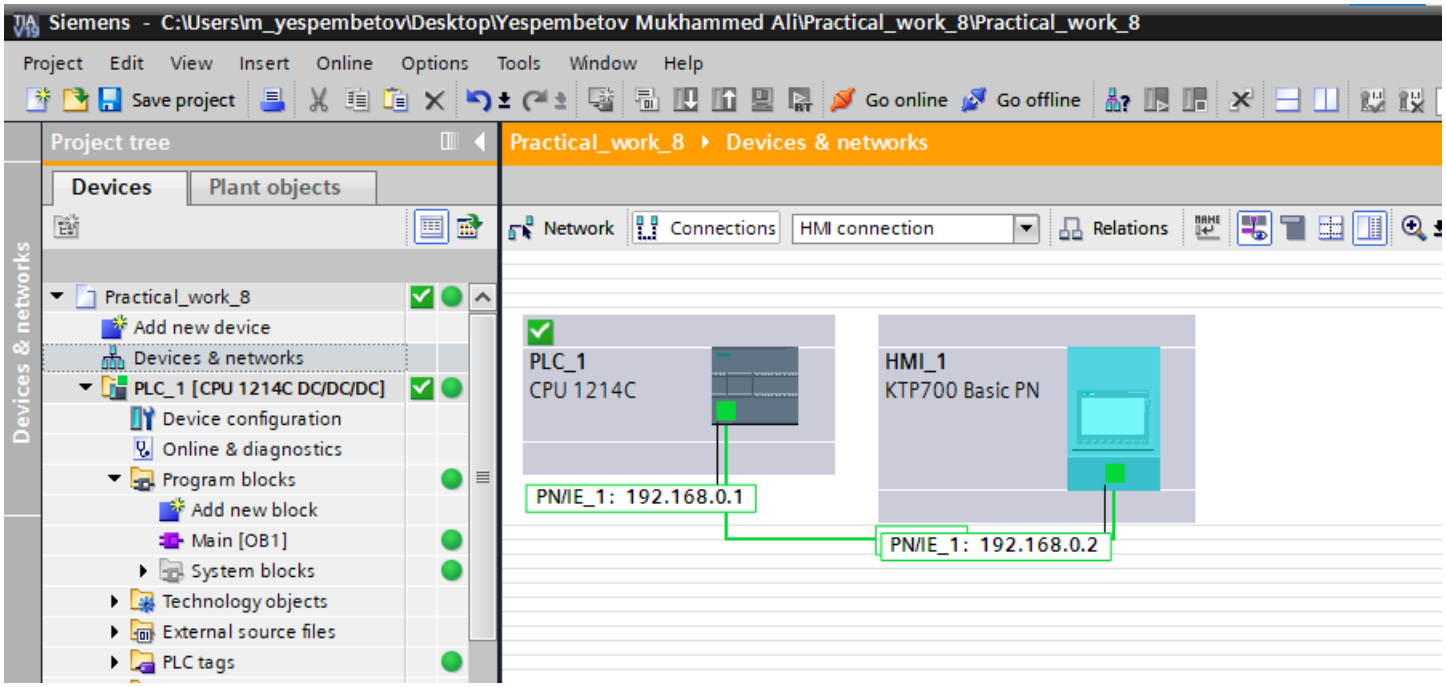
We are getting to checking results with simulation.



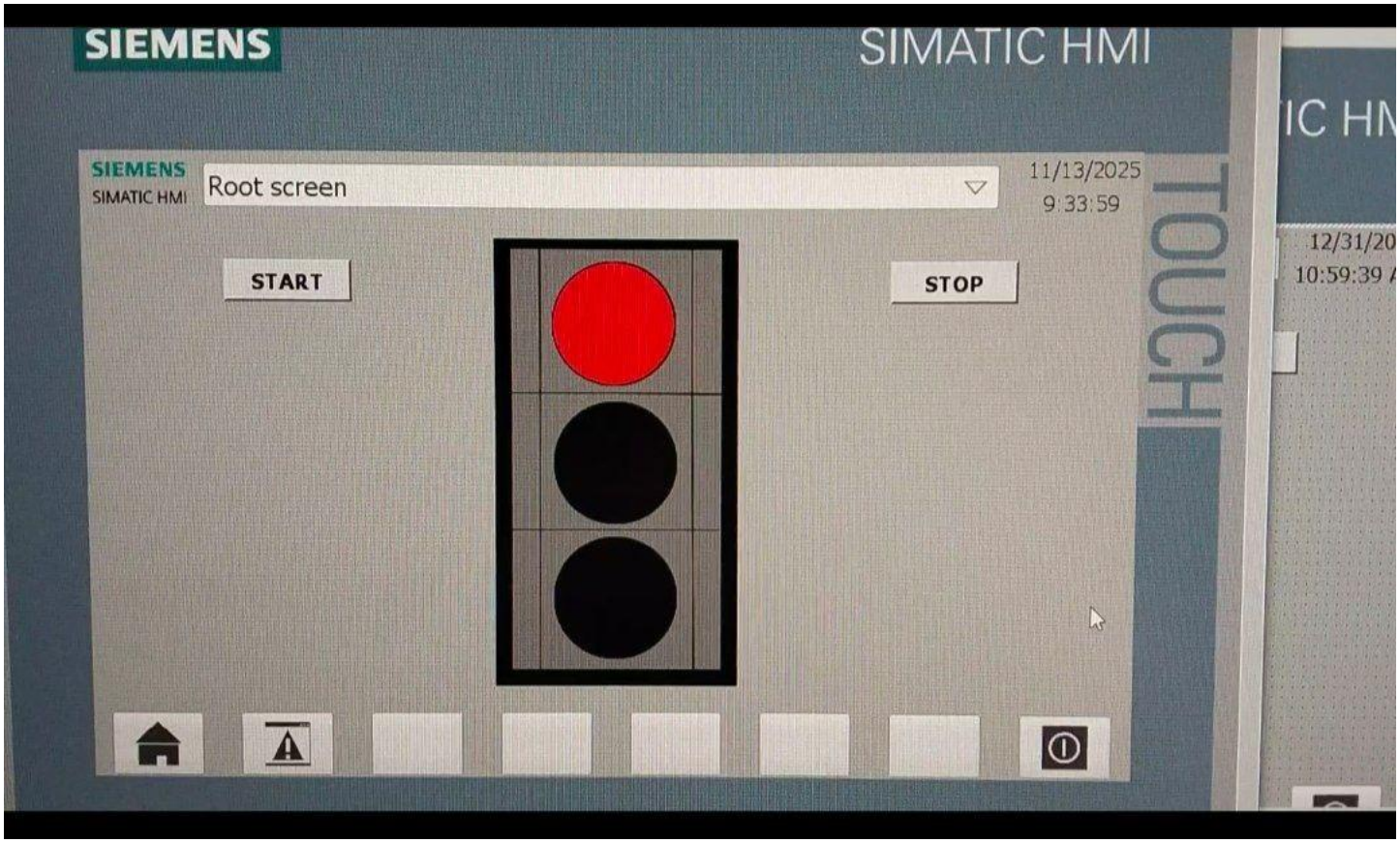




HMI should be connected with PLC (It must be connected with the IP address, check it.)



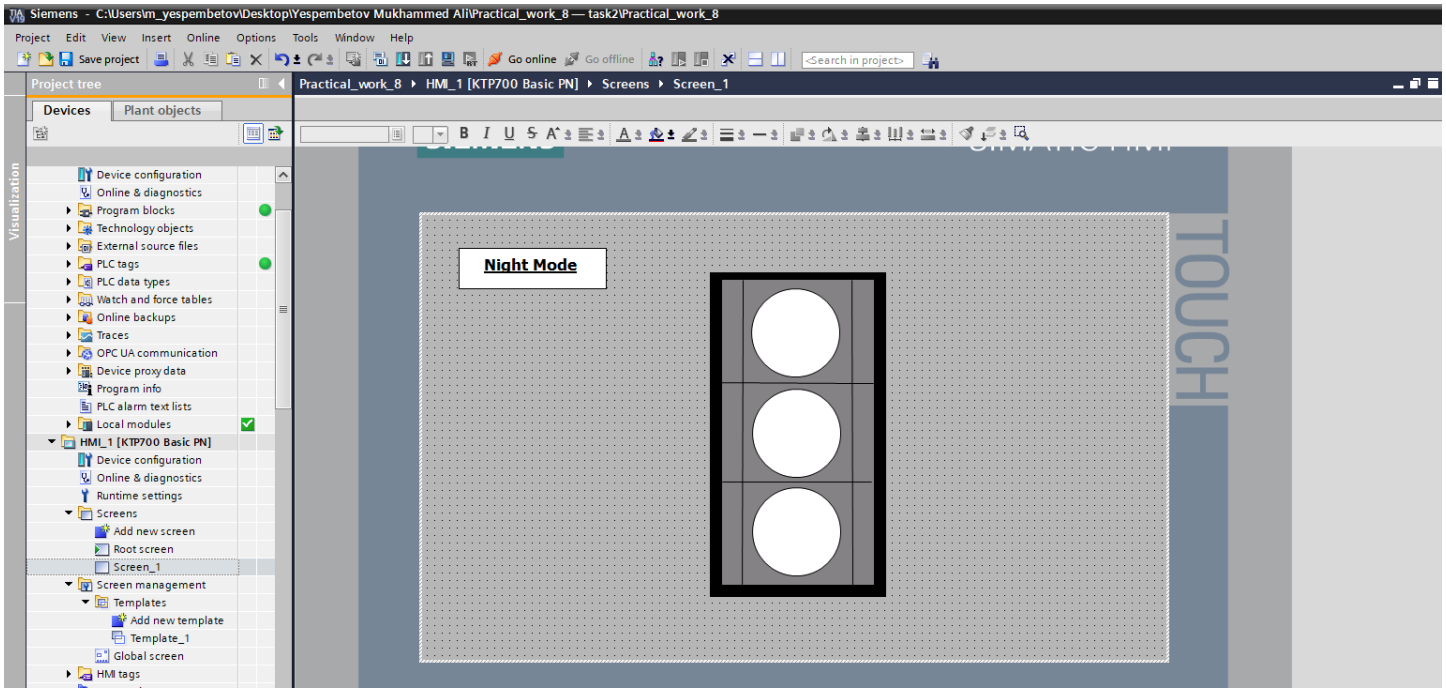
Here is the HMI view of our task. If you want to see the full animation, open the next file. We have included the video in the slide.



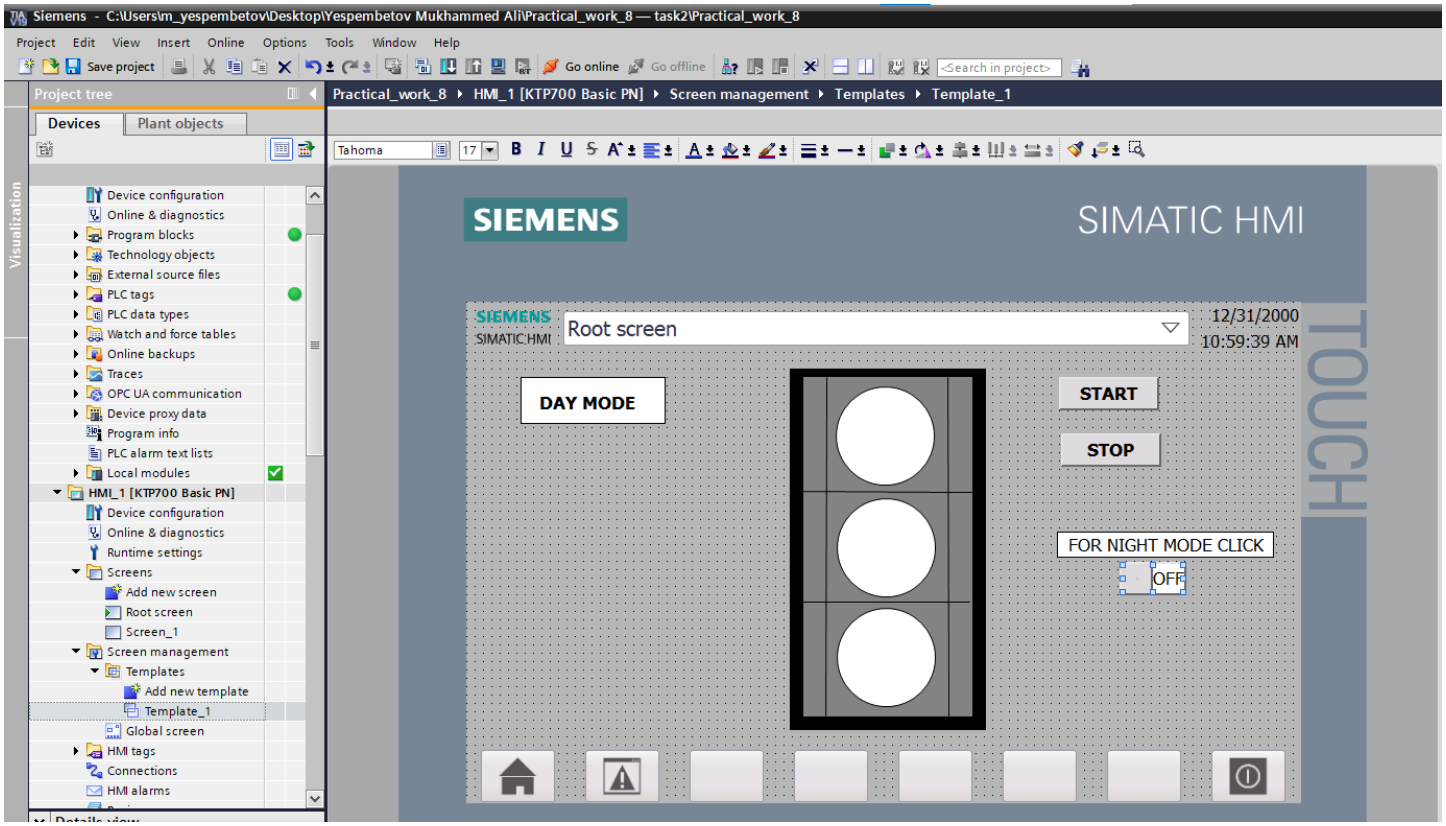
8.2 . Task for independent work:

Task 2. Develop improved program traffic light control program, adding the "flashing yellow signal" mode for night time operation. Realize automatic switching between day and night modes by a set time or via a control signal. Update the visualization on the HMI to show both modes of operation.

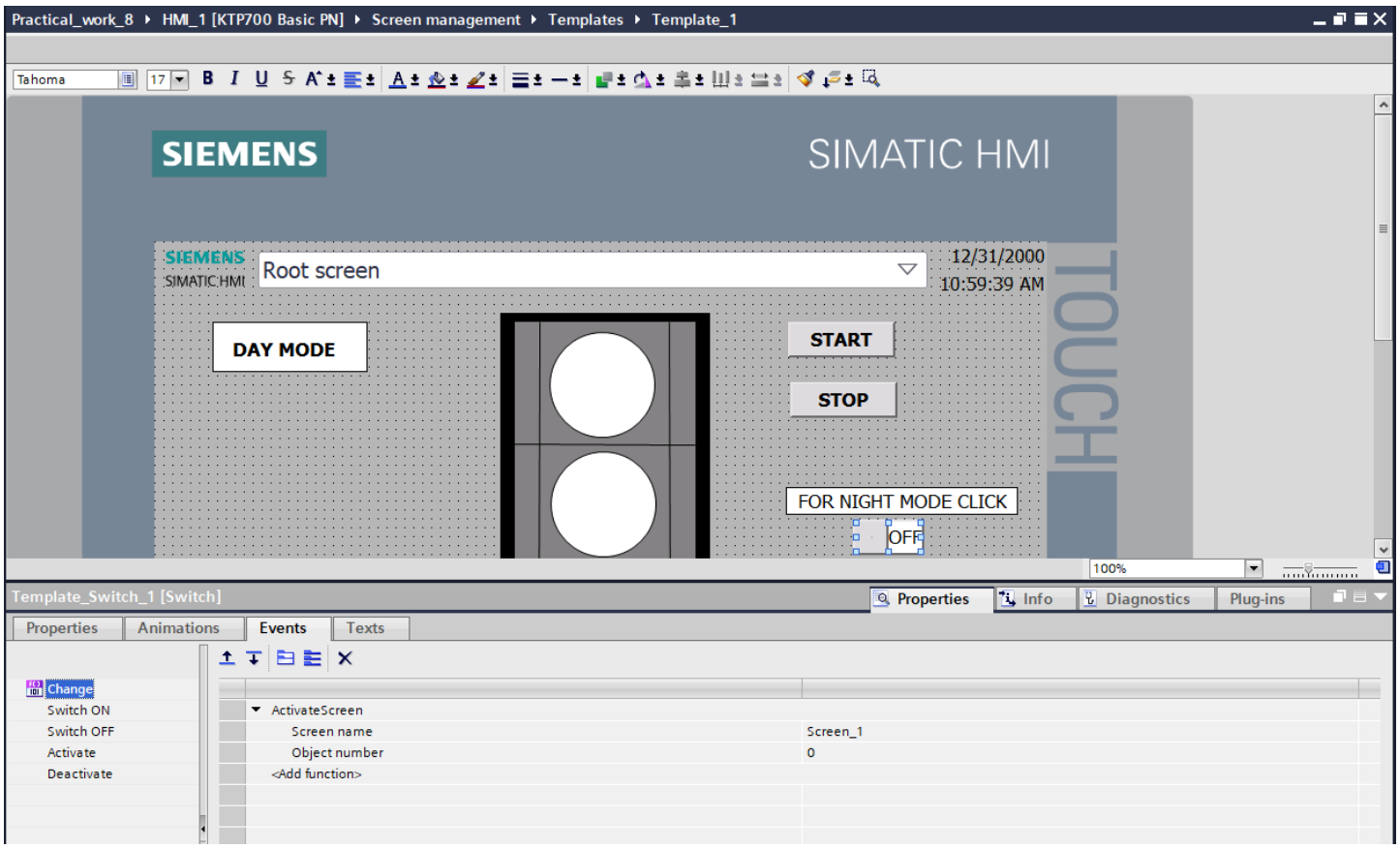
To display the day and night modes, we need to add a new screen. To do this, simply click the **“Add new screen”** button.



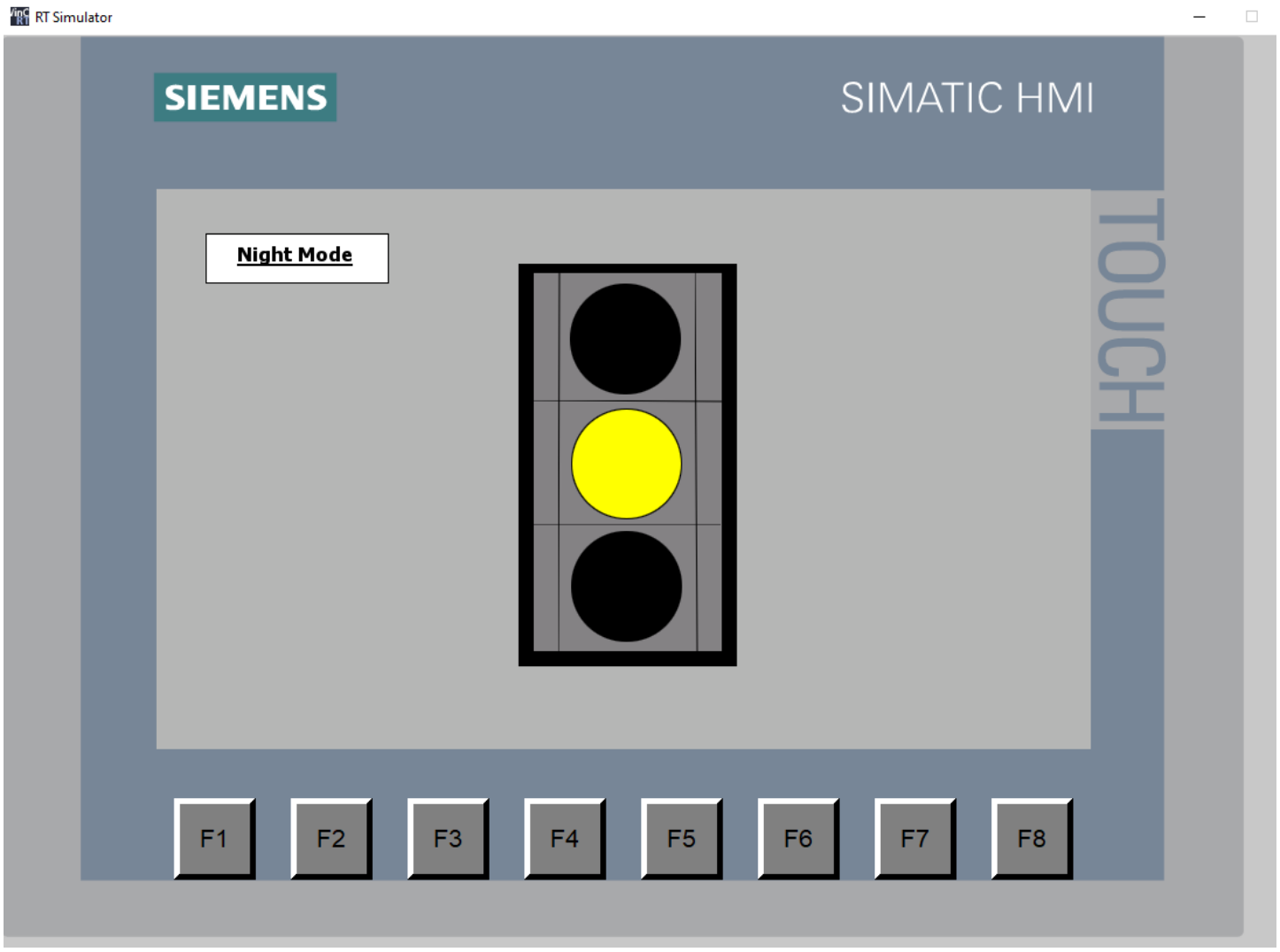
It is our day modes, here you can see (red, yellow, green) normally working of traffic lights



To link the two screens, we used the «ActivateScreen» function. After that, when the switch on our «Template_1» window is turned on, it connects to the «Screen_1» window.

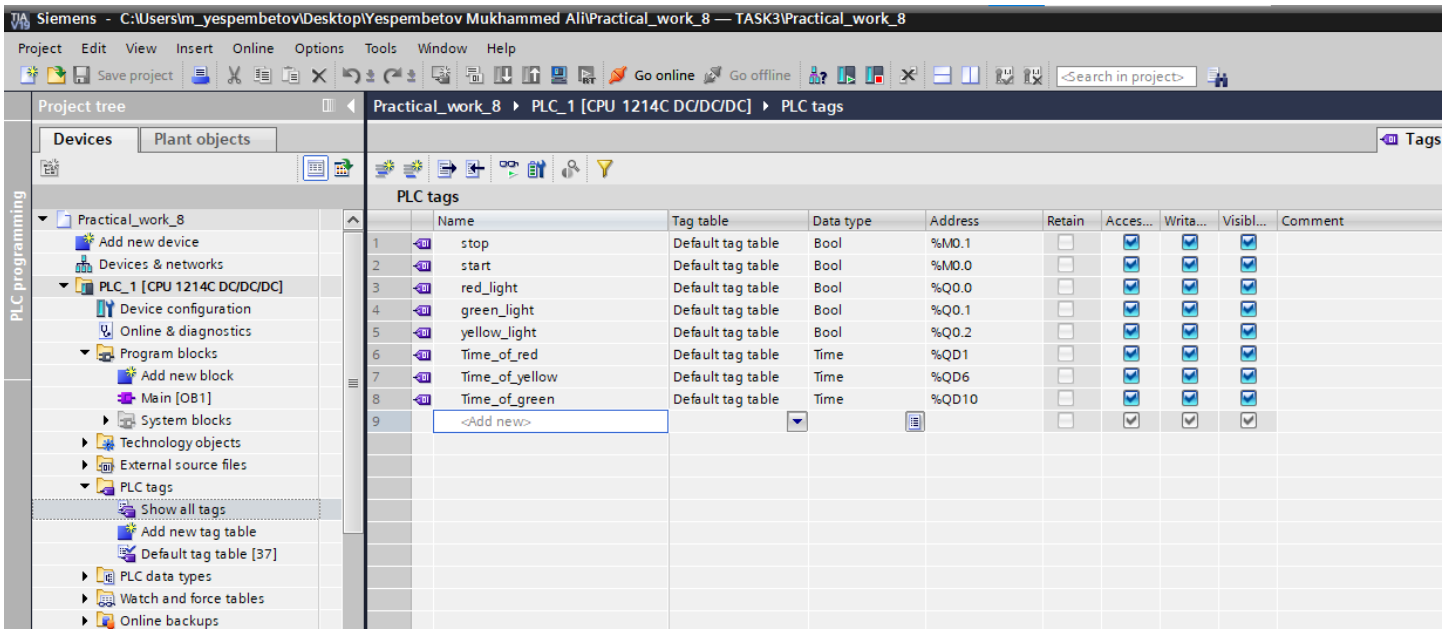


In night mode, the yellow lights blink.

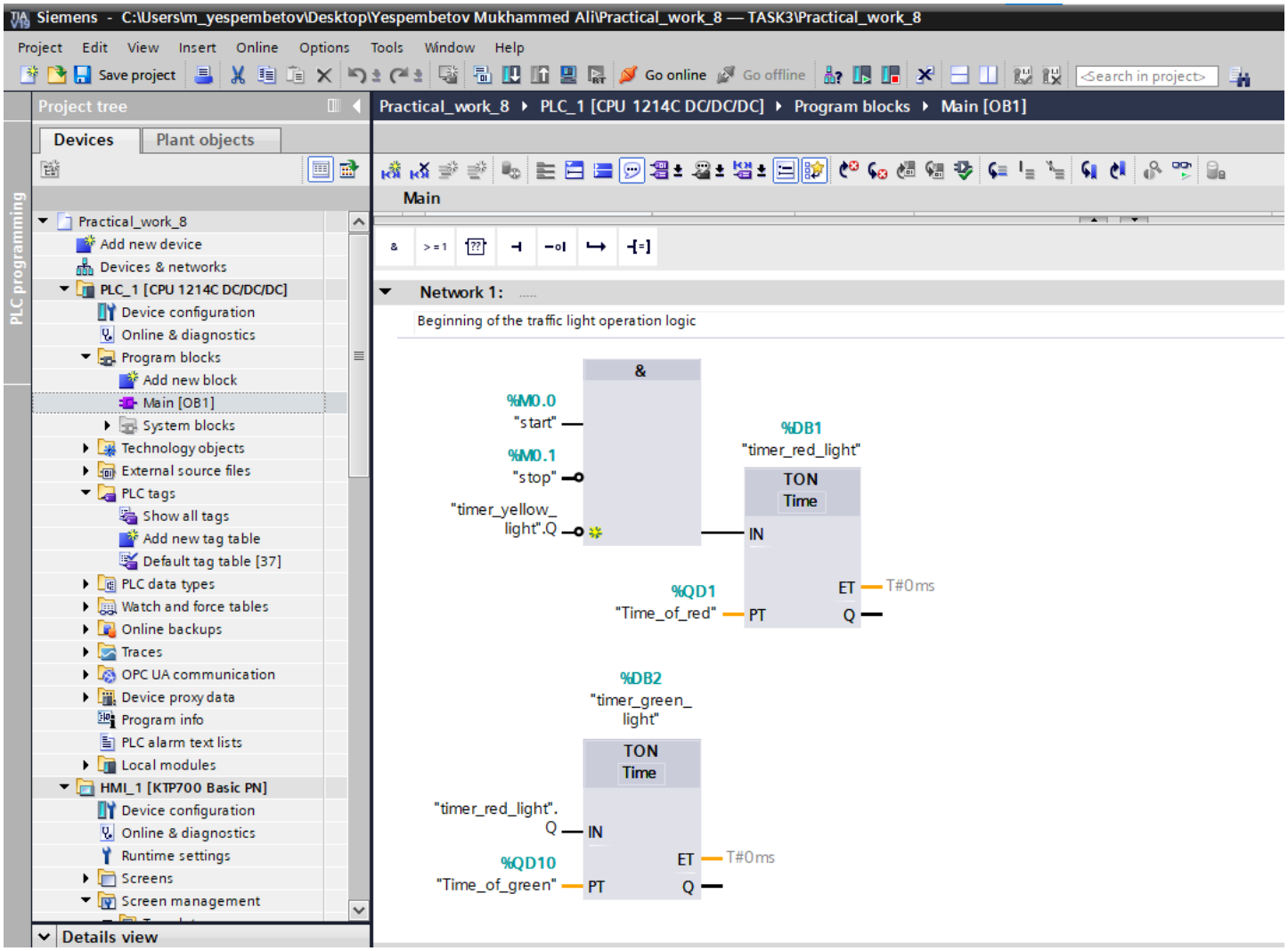


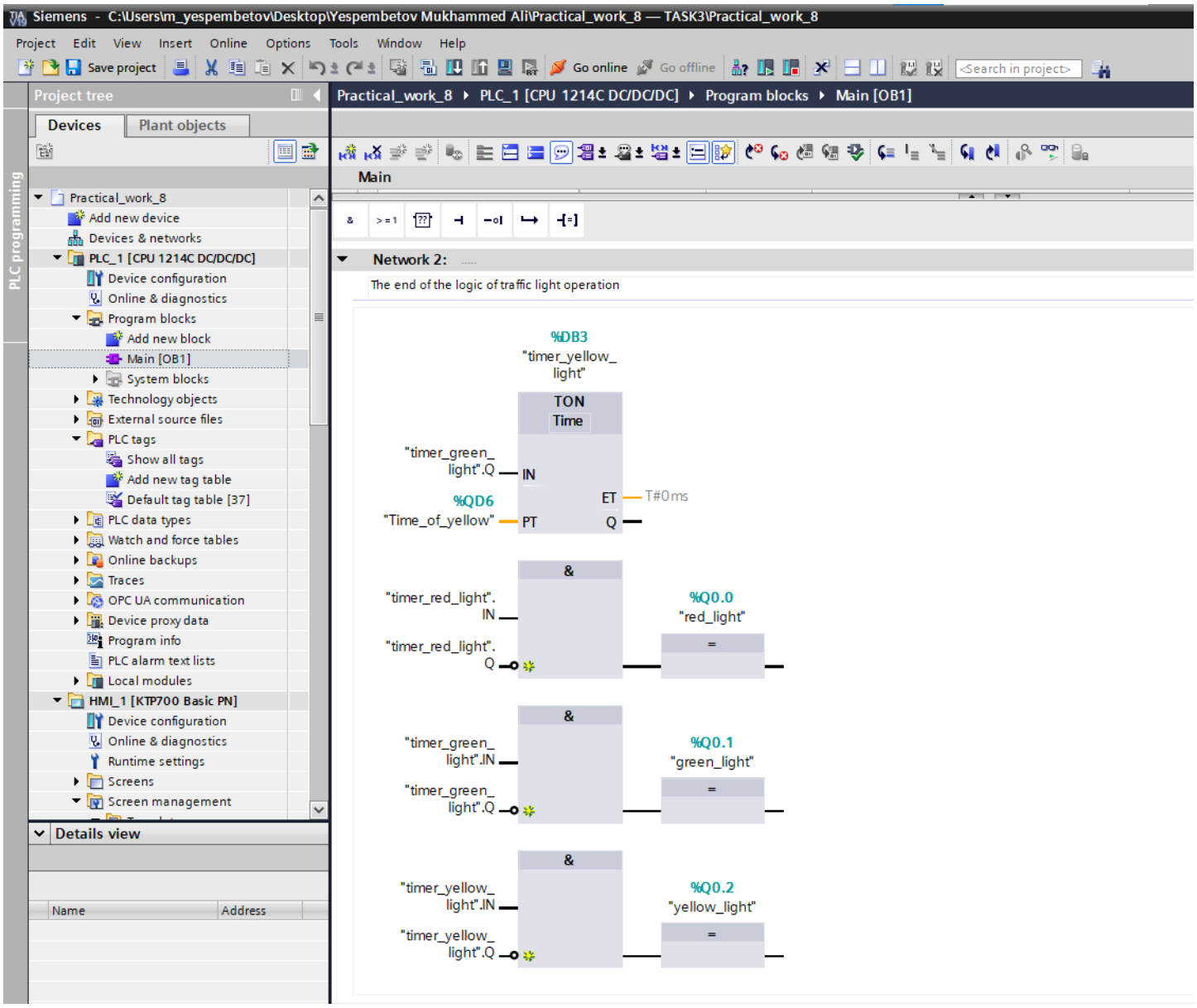
Task 3: Develop a traffic light control program with three signals (red, yellow, green), where the duration of each signal is set and can be changed from the HMI panel in real time. The switching of states should be done using timers. Additionally implement the pedestrian enquiry function: to activate it, two pedestrian buttons should be pressed simultaneously, after which the duration of the current signal is reduced to a minimum value.

To be able to change the light durations in the HMI, we create a new tag that accepts a time operation.

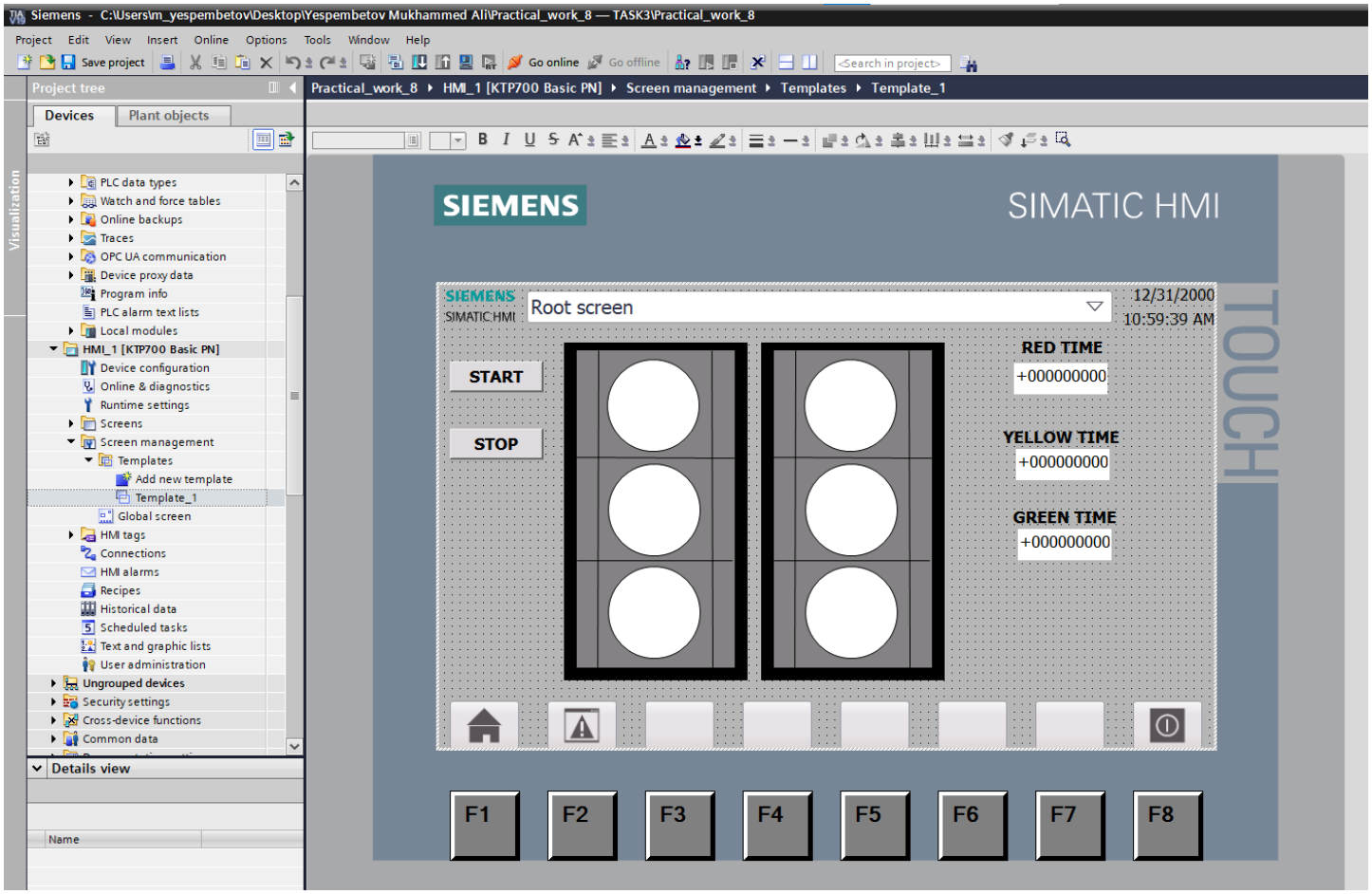


We place each tag in its proper position.

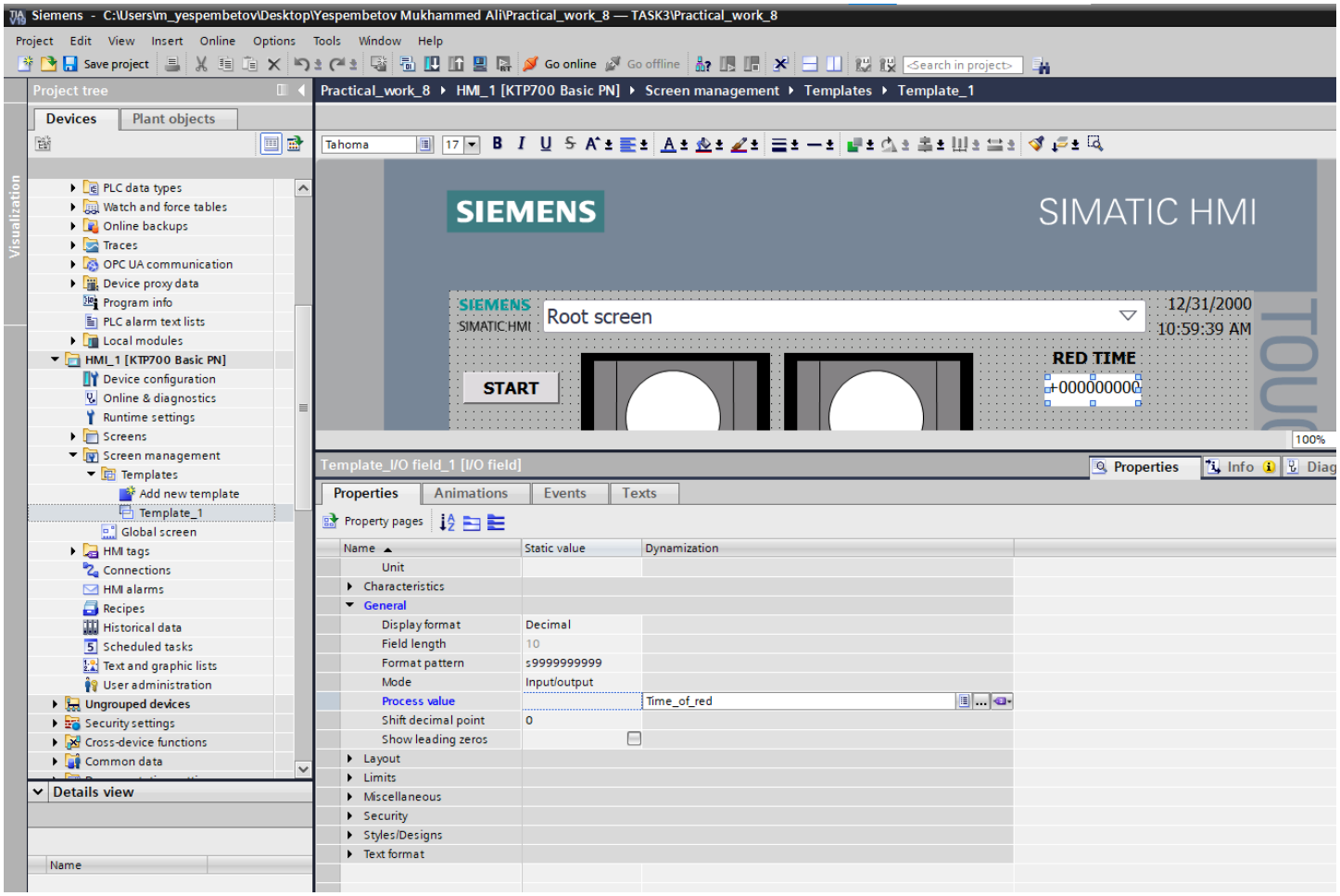


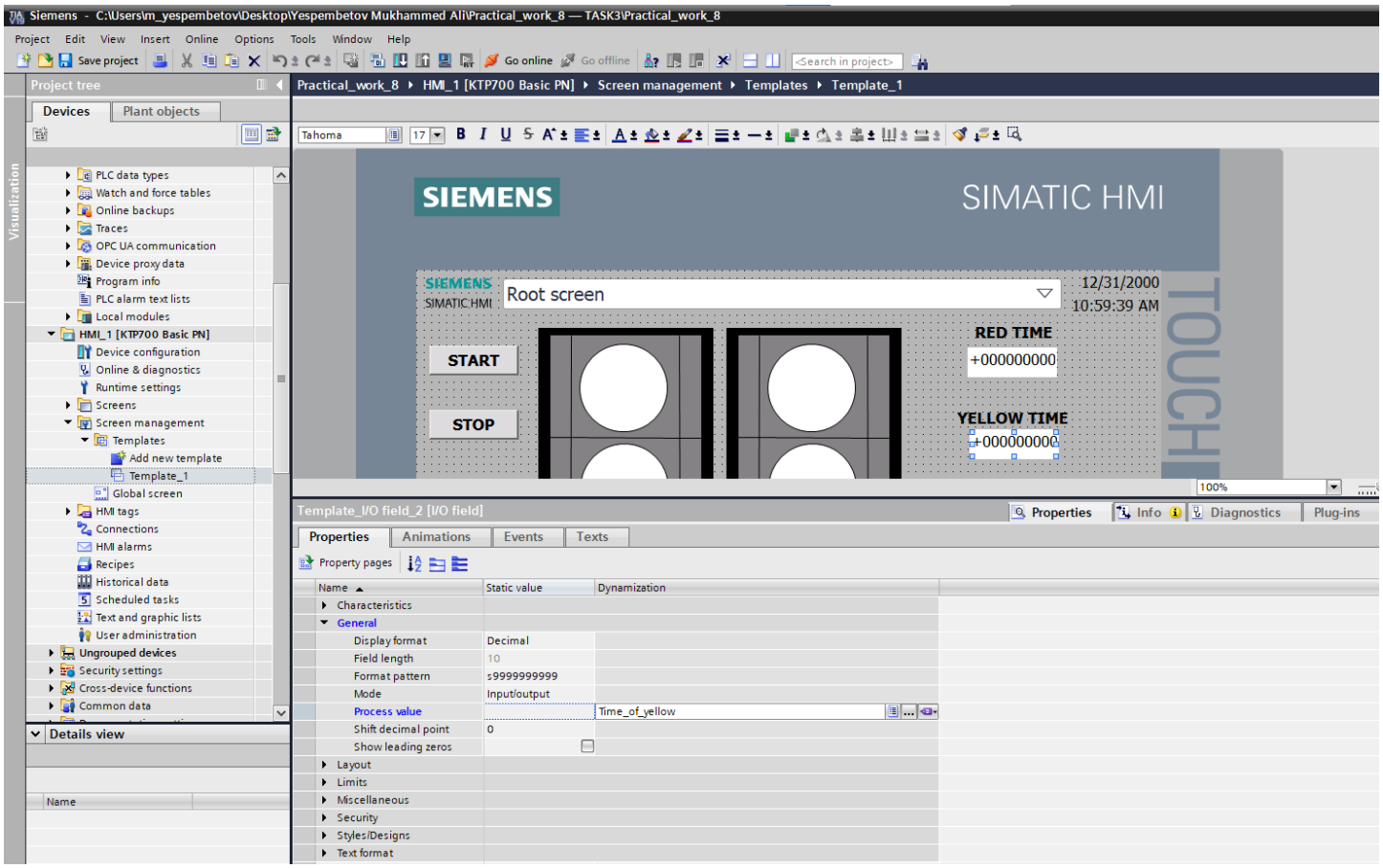

Siemens - C:\Users\m_yespembetov\Desktop\Yespembetov Mukhammed Ali\Practical_work_8 — TASK3\Practical_work_8
Project Edit View Insert Online Options Tools Window Help
Save project
Go online
Go offline
<Search in project>
Project tree
Devices
Plant objects
Practical_work_8
Add new device
Devices & networks
PLC_1 [CPU 1214C DC/DC/DC]
Device configuration
Online & diagnostics
Program blocks
Add new block
Main [OB1]
System blocks
Technology objects
External source files
PLC tags
Show all tags
Add new tag table
Default tag table [37]
PLC data types
Watch and force tables
Online backups
Traces
OPC UA communication
Device proxy data
Program info
PLC alarm text lists
Local modules
HMI_1 [KTP700 Basic PN]
Device configuration
Online & diagnostics
Runtime settings
Screens
Screen management
Details view
Name
Address
PLC programming
Practical_work_8 ▸ PLC_1 [CPU 1214C DC/DC/DC] ▸ Program blocks ▸ Main [OB1]
Main
Network 2:
The end of the logic of traffic light operation
%DB3
"timer_yellow_light"
TON
Time
"timer_green_light".Q
IN
%QD6
"Time_of_yellow"
PT
ET
T#0ms
Q
&
"timer_red_light".IN
"timer_red_light".Q
%Q0.0
"red_light"
=
&
"timer_green_light".IN
"timer_green_light".Q
%Q0.1
"green_light"
=
&
"timer_yellow_light".IN
"timer_yellow_light".Q
%Q0.2
"yellow_light"
=

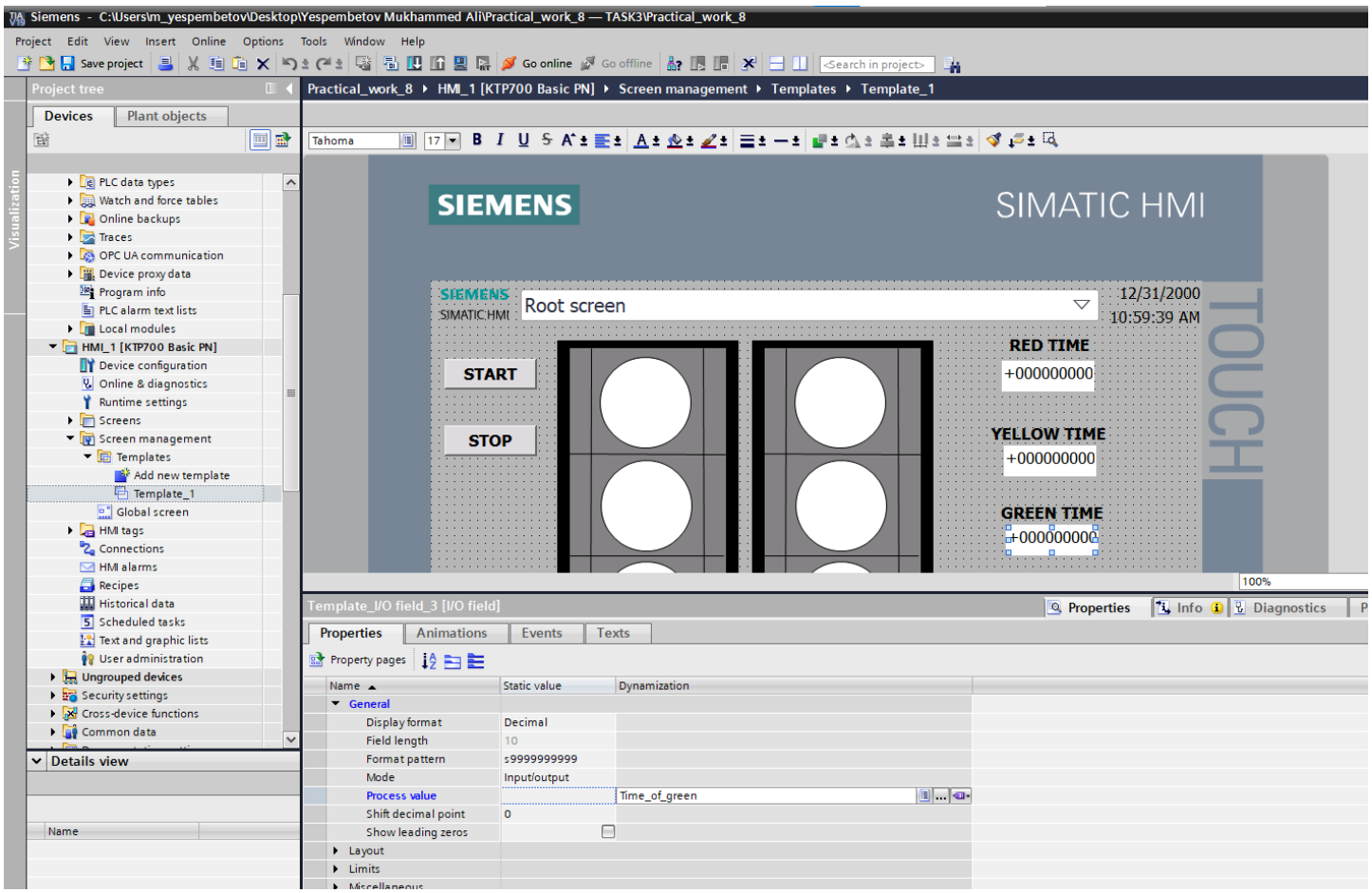
We will create it in the HMI.



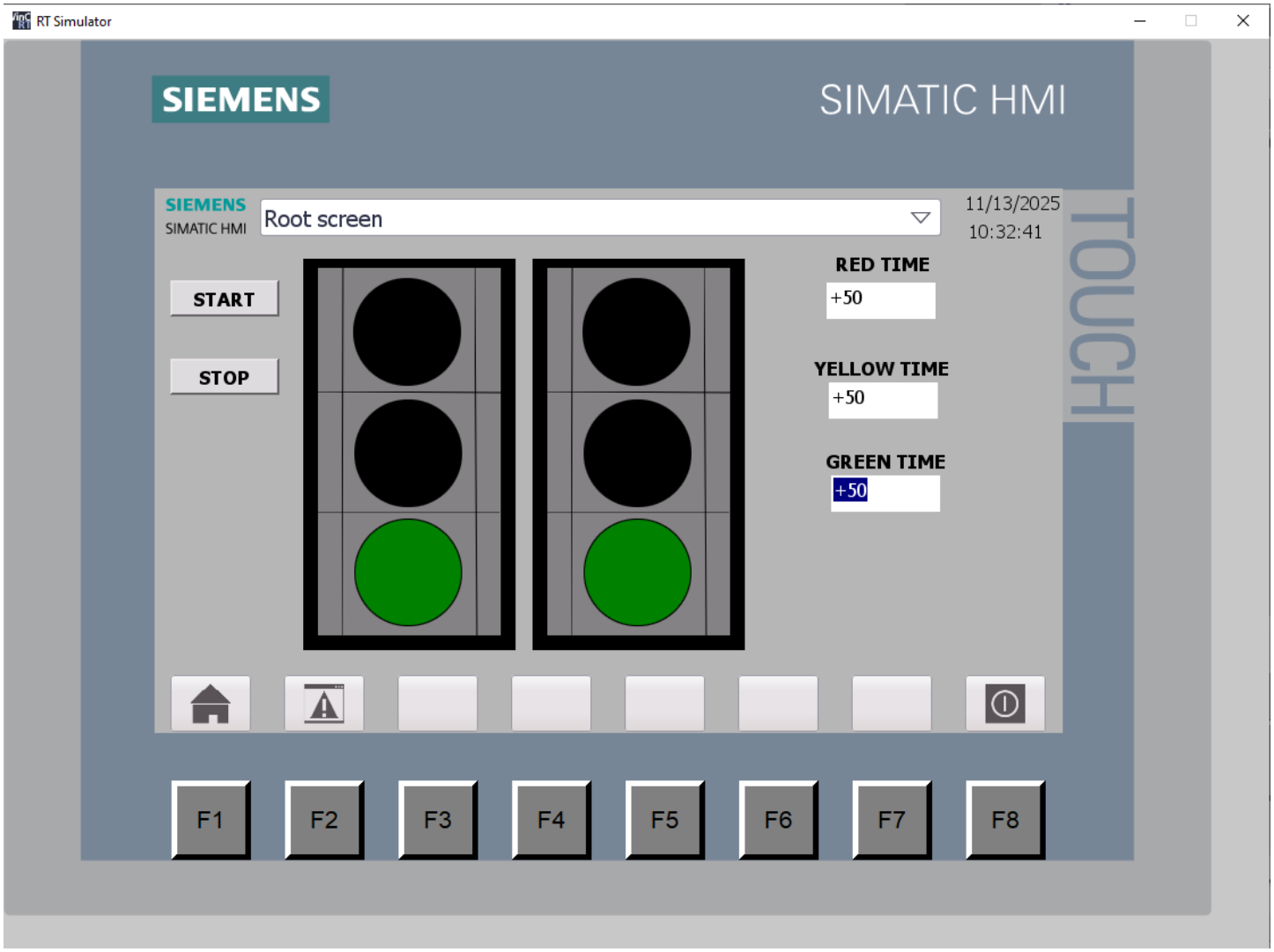
We link each timer to a PLC tag. To do this, go to **Properties → General → Process Value** and bind it to the PLC tag.



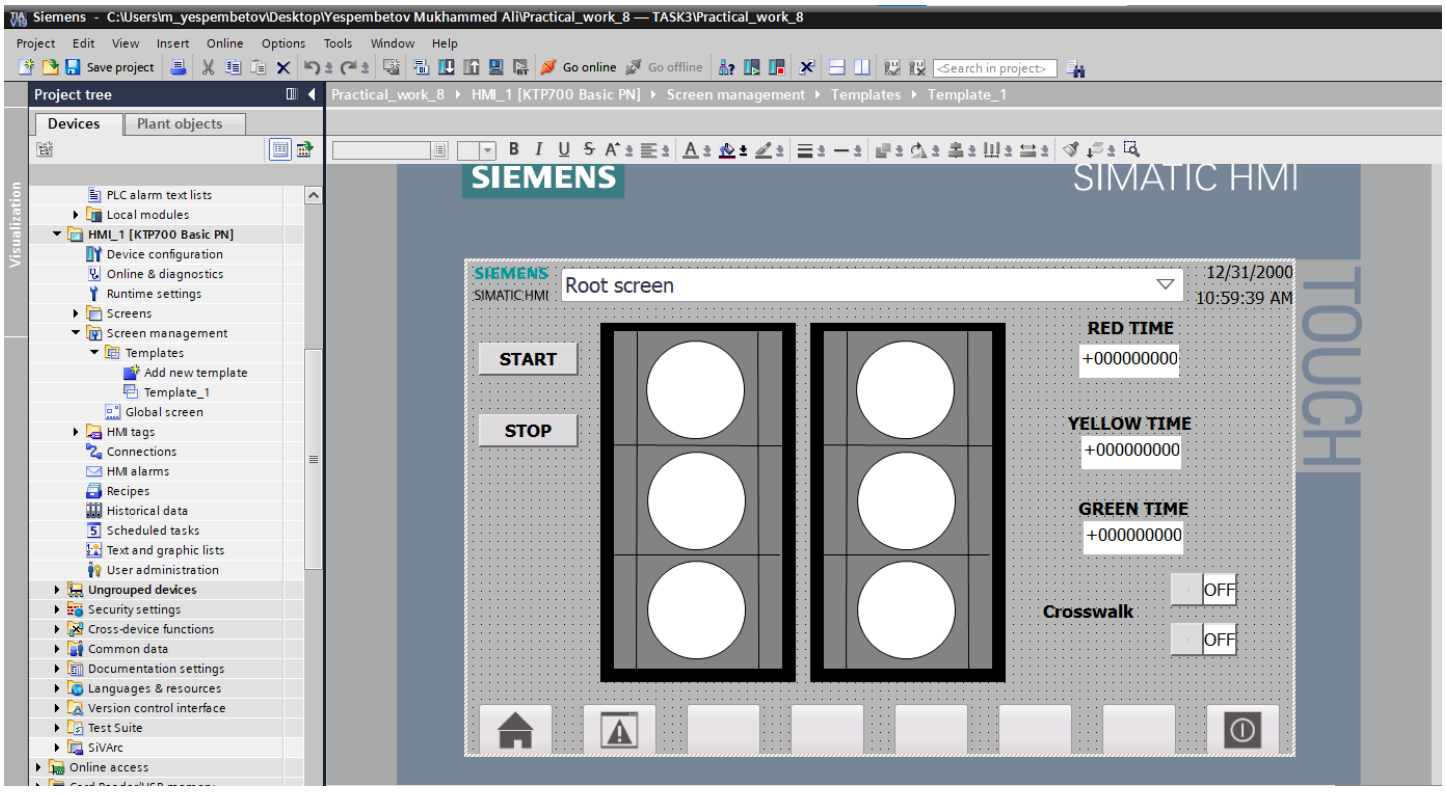

Siemens - C:\Users\m_yespembetov\Desktop\Yespembetov Mukhammed Ali\Practical_work_8 — TASK3\Practical_work_8
Project tree
Devices
Plant objects
Practical_work_8 ▸ HMI_1 [KTP700 Basic PN] ▸ Screen management ▸ Templates ▸ Template_1
SIEMENS
SIMATIC HMI
Root screen
12/31/2000
10:59:39 AM
START
STOP
RED TIME
+000000000
YELLOW TIME
+000000000
TOUCH
Template_I/O field_2 [I/O field]
Properties
Info
Diagnostics
Plug-ins
Animations
Events
Texts
Property pages
Name
Static value
Dynamization
Characteristics
General
Display format
Decimal
Field length
10
Format pattern
s9999999999
Mode
Input/output
Process value
Time_of_yellow
Shift decimal point
0
Show leading zeros
Layout
Limits
Miscellaneous
Security
Styles/Designs
Text format
Details view


Siemens - C:\Users\m_yespembetov\Desktop\Yespembetov Mukhammed Ali\Practical_work_8 — TASK3\Practical_work_8
Project tree
Devices
Plant objects
Practical_work_8 ▸ HMI_1 [KTP700 Basic PN] ▸ Screen management ▸ Templates ▸ Template_1
SIEMENS
SIMATIC HMI
Root screen
12/31/2000
10:59:39 AM
START
STOP
RED TIME
+000000000
YELLOW TIME
+000000000
GREEN TIME
+000000000
TOUCH
Template_I/O field_3 [I/O field]
Properties
Info
Diagnostics
Animations
Events
Texts
Property pages
Name
Static value
Dynamization
General
Display format
Decimal
Field length
10
Format pattern
s9999999999
Mode
Input/output
Process value
Time_of_green
Shift decimal point
0
Show leading zeros
Layout
Limits
Details view

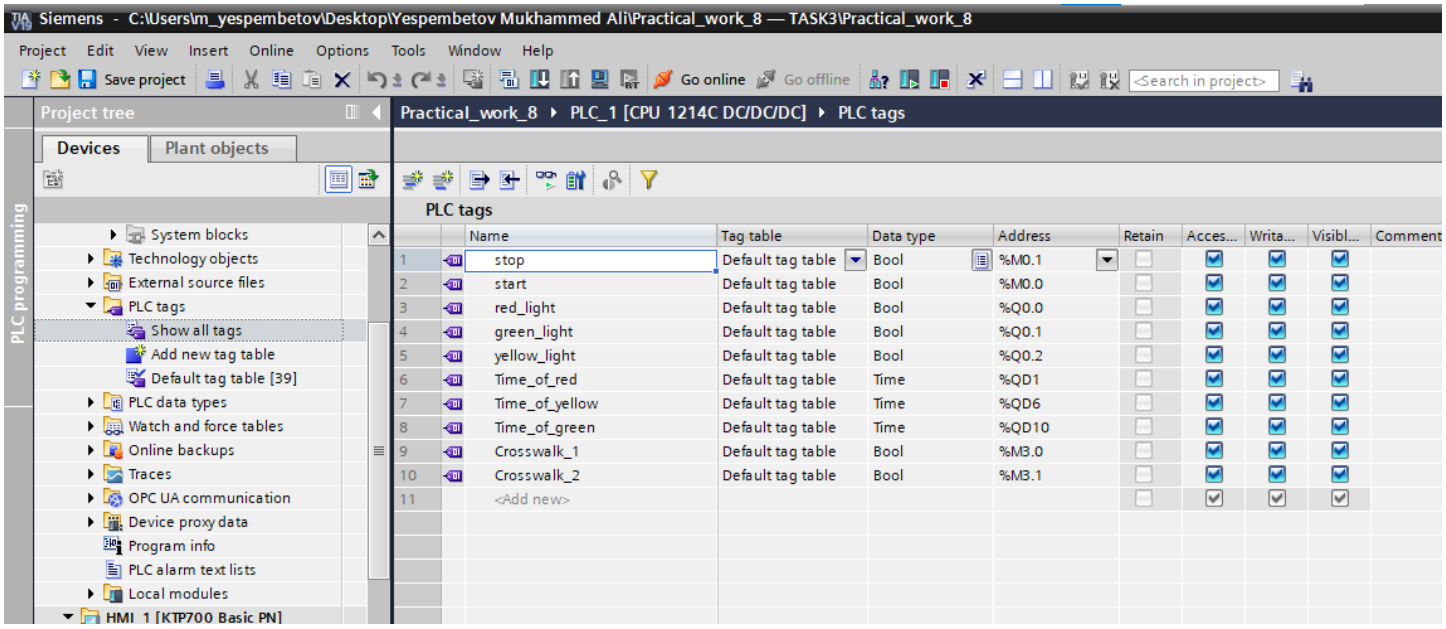
First, let's make sure this is working. You can see it in our file. It really works (the operator can set the desired time).



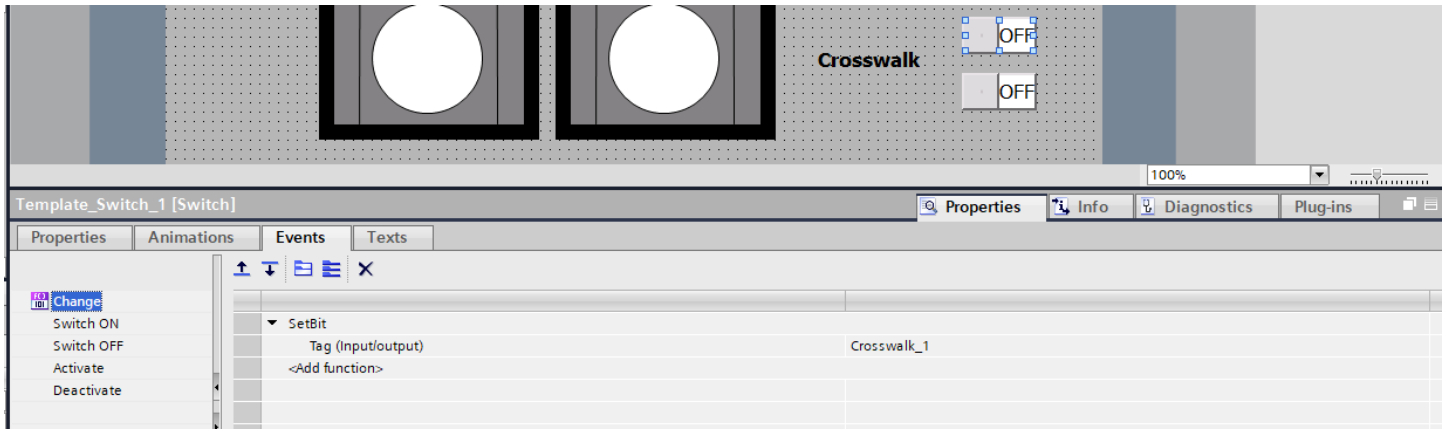
Now we need to add two pedestrian buttons. If either button is pressed, the time should be reduced to a minimum and stop (sorry, the red light should turn on—I didn't consider this before).



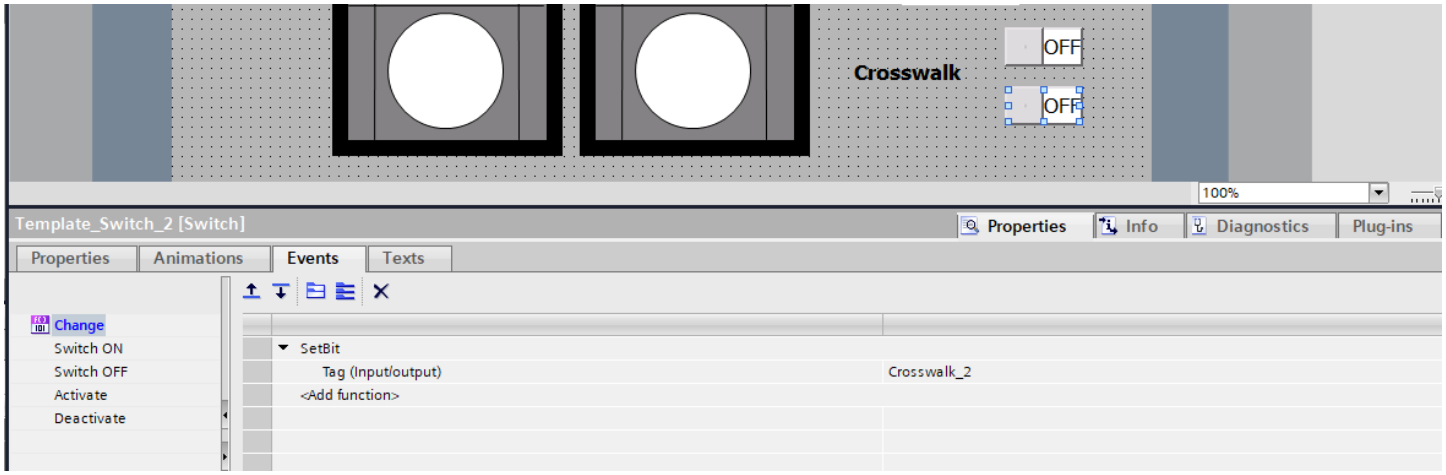
Here “Crosswalk_1” and “Crosswalk_2” are pedestrian buttons. We write PLC tags.



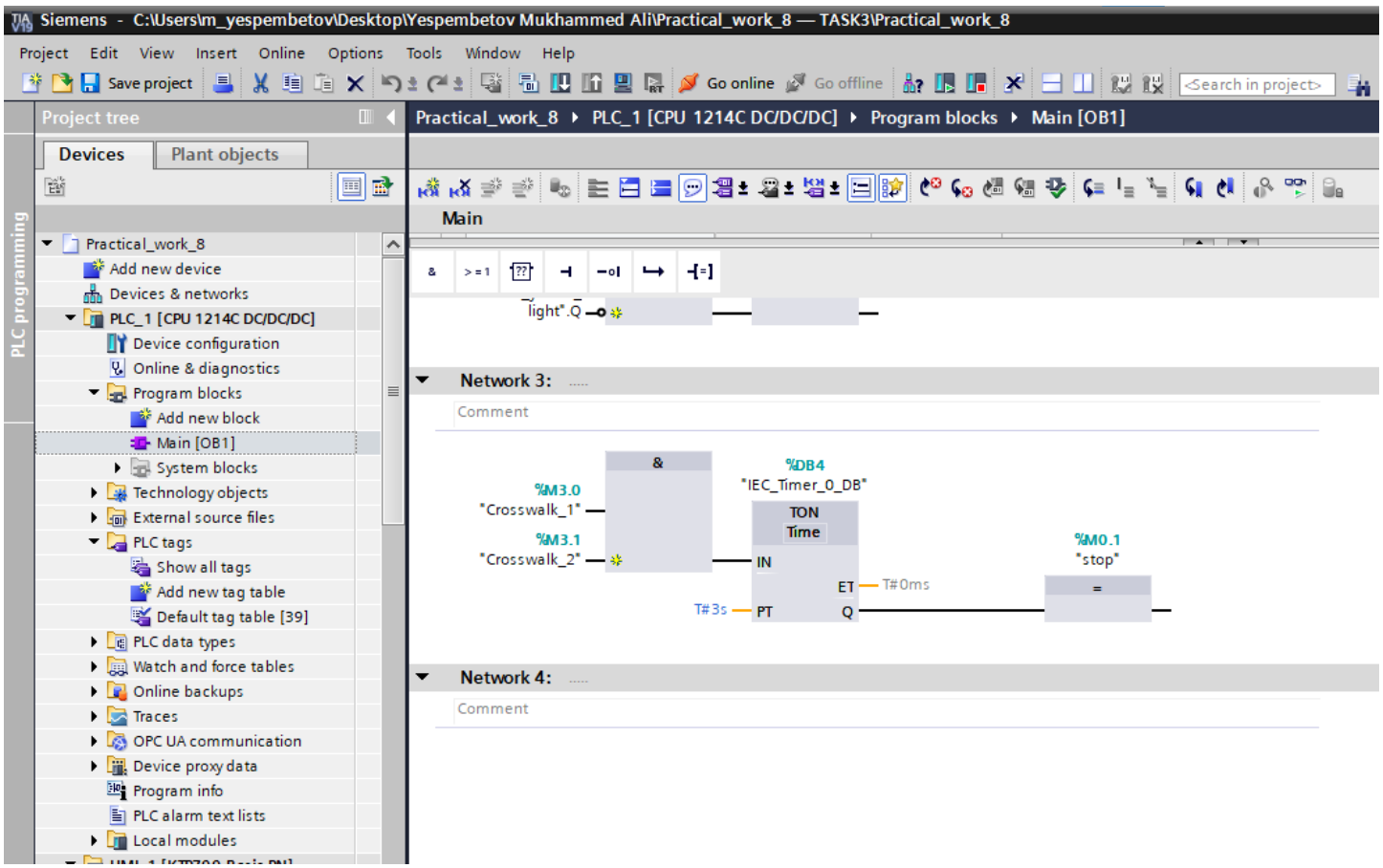
We link each button to a PLC tag.



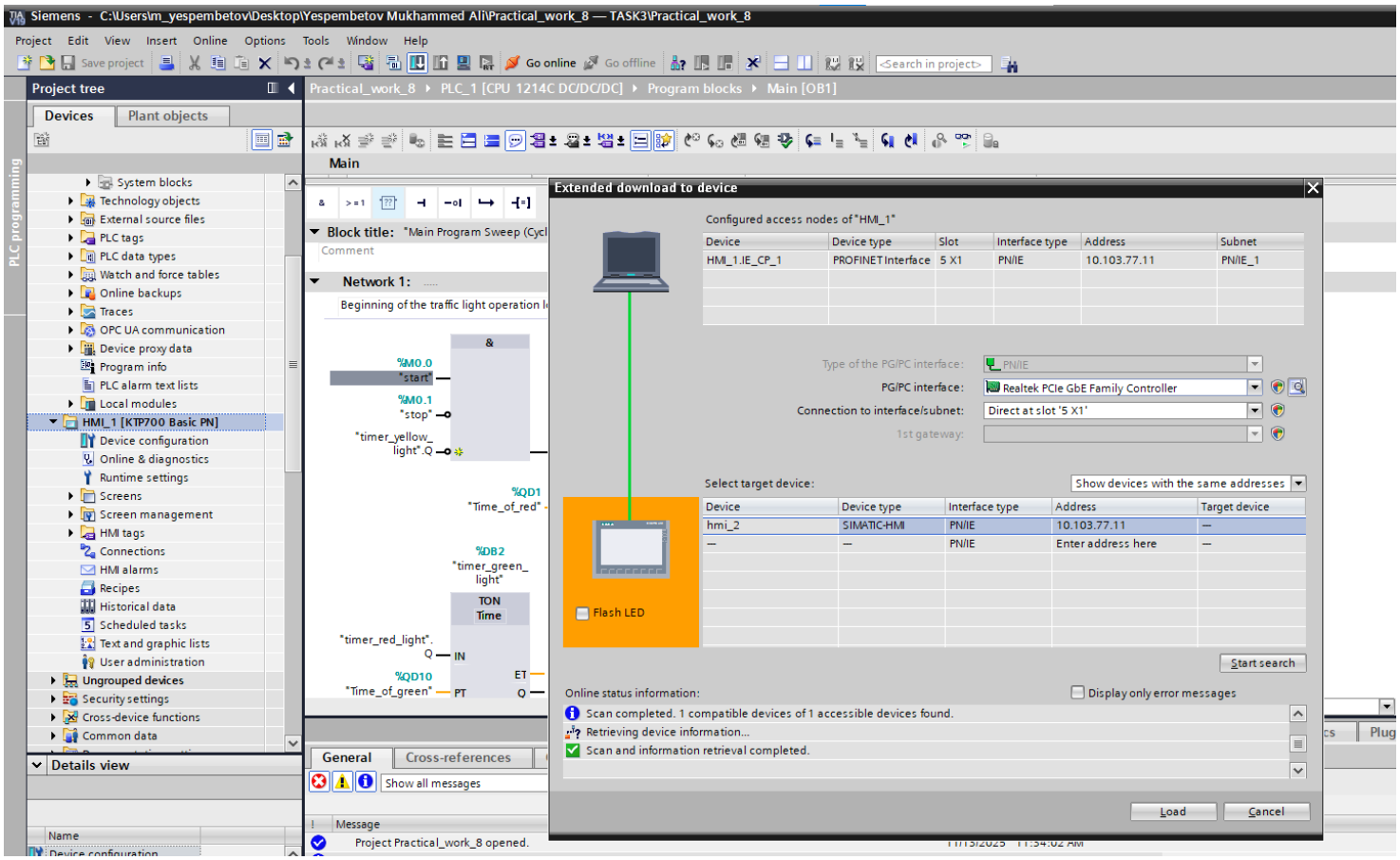
We link each button to a PLC tag.



Now we need to create the main logic in the Main block. I built a logic where the stop is activated 3 seconds after the buttons are pressed.



If we enter the IP address correctly, we can connect it to the offline simulation.



Control Questions

1) What is the FBD programming language and what standard does it belong to?

FBD (Function Block Diagram) is a graphical programming language used to create programs for PLCs (Programmable Logic Controllers). It belongs to the international standard IEC 61131-3.

2) What are the advantages of the FBD language compared to other IEC 61131-3 languages?

It shows logic visually with blocks and connections, so it is easier to understand.

Debugging is simpler because you can see how signals flow between blocks.

Good for complex control systems where many operations happen at the same time.

3) What are the basic elements of an FBD program?

Blocks – each block does a specific task (logical, arithmetic, control, etc.).

Connections – wires that link the blocks and transfer signals.

Networks – groups of blocks that form one part of the program.

Inputs and Outputs – where the data comes in and goes out of the program.

4) What is a Network in FBD and what is its role?

A Network is a section of an FBD program that contains one or more blocks and their connections. Its role is to organize the program into smaller parts, making it easier to read and understand. Each network usually ends with an output block that shows the result.

5) What types of blocks can be used in FBD? Give examples.

Logical blocks – for decisions, e.g., AND, OR, NOT.

Arithmetic blocks – for calculations, e.g., ADD, SUBTRACT, MULTIPLY.

Control blocks – for controlling actions, e.g., TIMER, COUNTER.

Special blocks – for specific functions, e.g., MOVE, COMPARISON.

Conclusion

In this practical work, we learned the basic principles of programming in **Function Block Diagram (FBD)** language. FBD allows us to create programs for PLCs using visual blocks, which makes understanding and debugging the program much easier than using text-based languages. By working with FBD, we became familiar with the structure of a program, including blocks, connections, and networks.

We studied different types of blocks, such as logical, arithmetic, and control blocks, and learned how to use them to perform various operations. This practice helped us understand how signals move through the program and how outputs are calculated from inputs. We also learned the importance of networks in organizing the program and making it readable.

During the lab, we gained experience in creating, testing, and debugging simple programs in **TIA Portal** for the Siemens S7-1200 controller. This gave us confidence in using FBD to solve real control tasks and allowed us to see the practical application of theoretical knowledge.

Overall, this work improved our understanding of PLC programming and showed us that FBD is a powerful tool for designing control systems. It is especially useful for beginners and for complex systems where visual representation of logic makes programming clearer and faster.